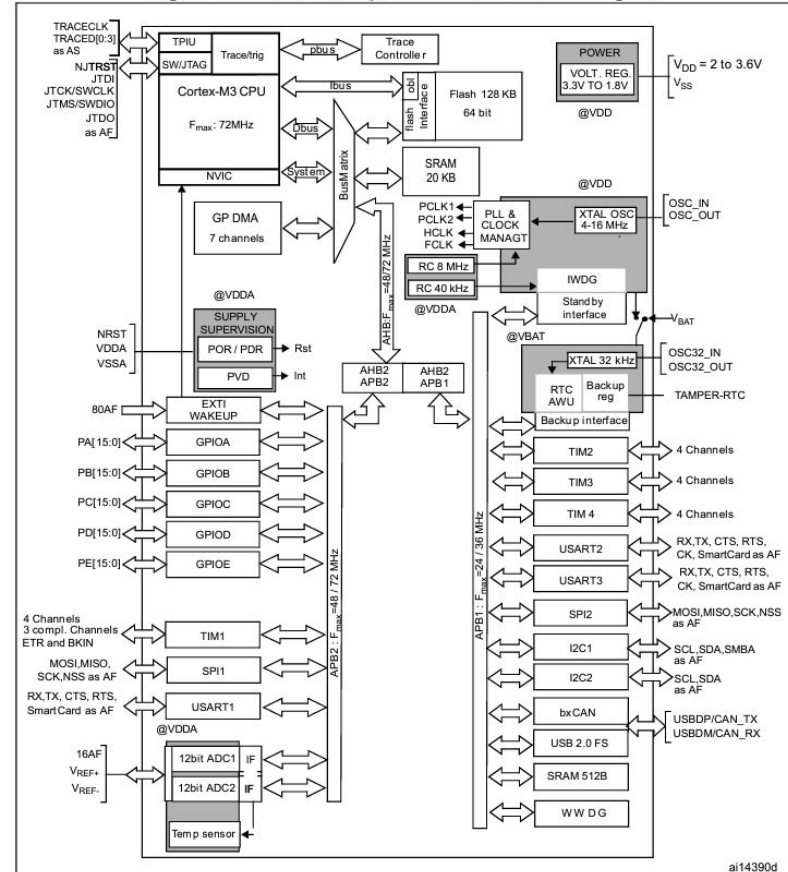


Periféricos

Periféricos SMTF103x8/B

Figure 1. STM32F103xx performance line block diagram



19.1 IWDG introduction

The devices have two embedded watchdog peripherals which offer a combination of high safety level, timing accuracy and flexibility of use. Both watchdog peripherals (Independent and Window) serve to detect and resolve malfunctions due to software failure, and to trigger system reset or an interrupt (window watchdog only) when the counter reaches a given timeout value.

The independent watchdog (IWDG) is clocked by its own dedicated low-speed clock (LSI) and thus stays active even if the main clock fails. The window watchdog (WWDG) clock is prescaled from the APB1 clock and has a configurable time-window that can be programmed to detect abnormally late or early application behavior.

The IWDG is best suited to applications which require the watchdog to run as a totally independent process outside the main application, but have lower timing accuracy constraints. The WWDG is best suited to applications which require the watchdog to react within an accurate timing window. For further information on the window watchdog, refer to [Section 20 on page 500](#).

19.2 IWDG main features

- Free-running downcounter
- clocked from an independent RC oscillator (can operate in Standby and Stop modes)
- Reset (if watchdog activated) when the downcounter value of 0x000 is reached

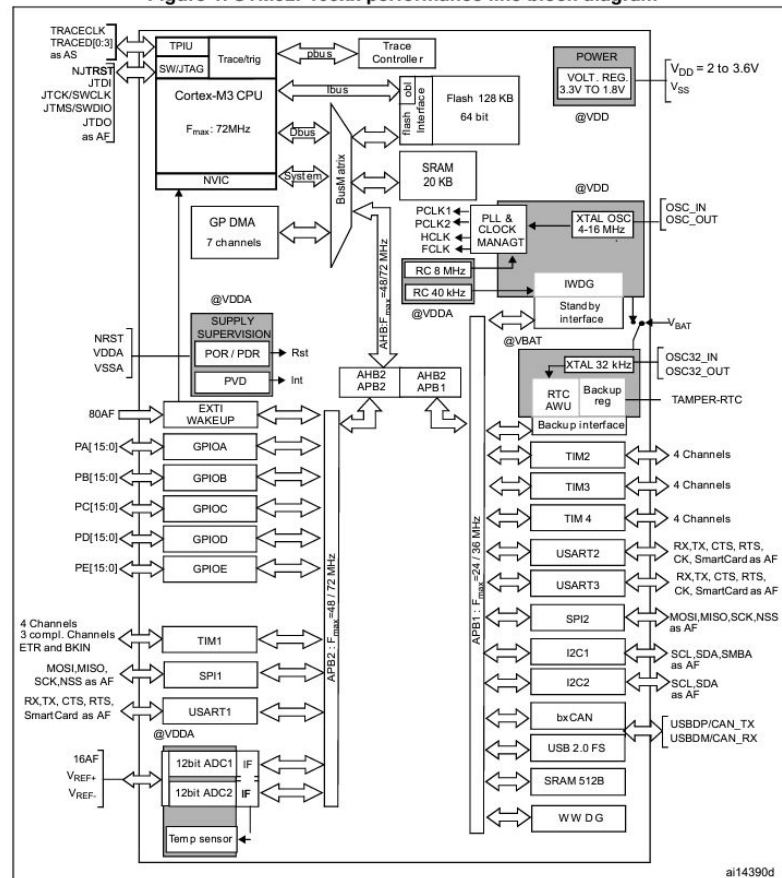
20.1 WWDG introduction

The window watchdog is used to detect the occurrence of a software fault, usually generated by external interference or by unforeseen logical conditions, which causes the application program to abandon its normal sequence. The watchdog circuit generates an MCU reset on expiry of a programmed time period, unless the program refreshes the contents of the downcounter before the T6 bit becomes cleared. An MCU reset is also generated if the 7-bit downcounter value (in the control register) is refreshed before the downcounter has reached the window register value. This implies that the counter must be refreshed in a limited window.

20.2 WWDG main features

- Programmable free-running downcounter
- Conditional reset
 - Reset (if watchdog activated) when the downcounter value becomes less than 0x40
 - Reset (if watchdog activated) if the downcounter is reloaded outside the window (see [Figure 184](#))
- Early wakeup interrupt (EWI): triggered (if enabled and the watchdog activated) when the downcounter is equal to 0x40.

Figure 1. STM32F103xx performance line block diagram



ai14390d

5.1.2 Battery backup domain

To retain the content of the Backup registers and supply the RTC function when V_{DD} is turned off, V_{BAT} pin can be connected to an optional standby voltage supplied by a battery or by another source.

The V_{BAT} pin powers the RTC unit, the LSE oscillator and the PC13 to PC15 I/Os, allowing the RTC to operate even when the main digital supply (V_{DD}) is turned off. The switch to the V_{BAT} supply is controlled by the Power Down Reset embedded in the Reset block.

6.1 BKP introduction

The backup registers are forty two 16-bit registers for storing 84 bytes of user application data.

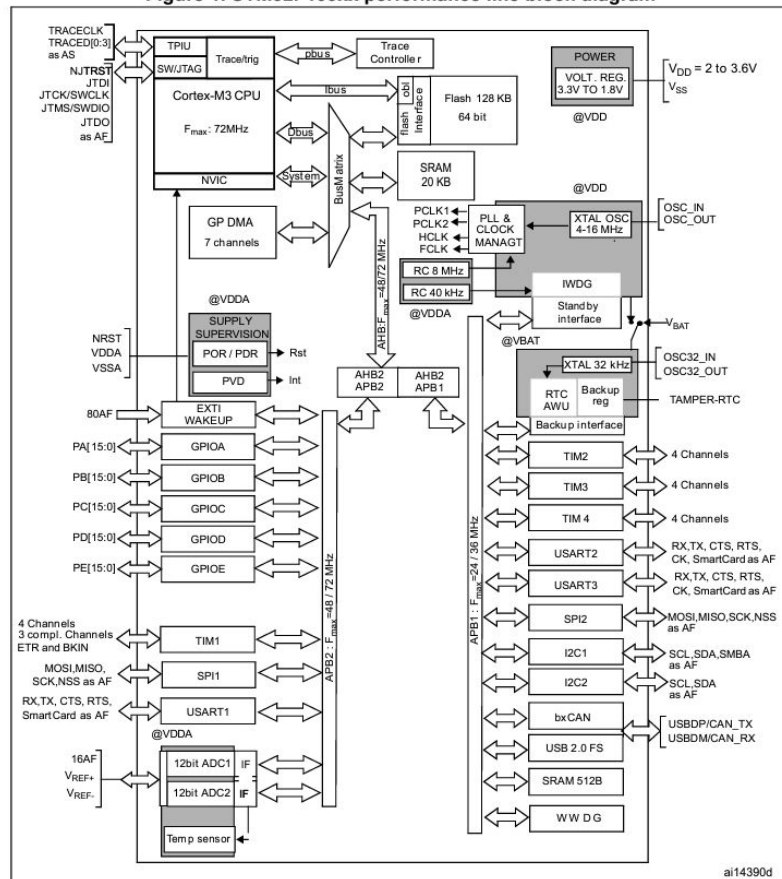
They are implemented in the backup domain that remains powered on by V_{BAT} when the V_{DD} power is switched off. They are not reset when the device wakes up from Standby mode or by a system reset or power reset.

In addition, the BKP control registers are used to manage the Tamper detection feature and RTC calibration.

After reset, access to the Backup registers and RTC is disabled and the Backup domain (BKP) is protected against possible parasitic write access. To enable access to the Backup registers and the RTC, proceed as follows:

- enable the power and backup interface clocks by setting the PWREN and BKPEN bits in the RCC_APB1ENR register
- set the DBP bit in the Power control register (PWR_CR) to enable access to the Backup registers and RTC.

Figure 1. STM32F103xx performance line block diagram



11.1 ADC introduction

The 12-bit ADC is a successive approximation analog-to-digital converter. It has up to 18 multiplexed channels allowing it measure signals from sixteen external and two internal sources. A/D conversion of the various channels can be performed in single, continuous, scan or discontinuous mode. The result of the ADC is stored in a left-aligned or right-aligned 16-bit data register.

The analog watchdog feature allows the application to detect if the input voltage goes outside the user-defined high or low thresholds.

The ADC input clock is generated from the PCLK2 clock divided by a prescaler and it must not exceed 14 MHz, refer to [Figure 8](#) for low-, medium-, high- and XL-density devices, and to [Figure 11](#) for connectivity line devices.

The Cortex-M4 based STM32F373 product line integrates 16-bit sigma-delta ADCs, making the devices perfect fit for all kinds of sensor applications requiring high precision measurement together with more demanding signal processing.

The devices are equipped with:

- 3x 16-bit sigma-delta ADCs, with up to 21 single or 11 differential channels and seven programmable gains per channel
- 12-bit ADCs with 1 MSPS
- Up to 2x comparators
- Up to 3x 12-bit DACs
- USB FS and CAN 2.0B communication interfaces



ADC12DJ5200-SEP
SBAS4908 – OCTOBER 2023

ADC12DJ5200-SEP 10.4-GSPS Single-Channel or 5.2-GSPS Dual-Channel, 12-bit, RF-Sampling Analog-to-Digital Converter (ADC)



ADS1299, ADS1299-4, ADS1299-6
SBAS499C – JULY 2012 – REVISED JANUARY 2017

ADS1299-x Low-Noise, 4-, 6-, 8-Channel, 24-Bit, Analog-to-Digital Converter for EEG and Biopotential Measurements

Figure 1. STM32F103xx performance line block diagram

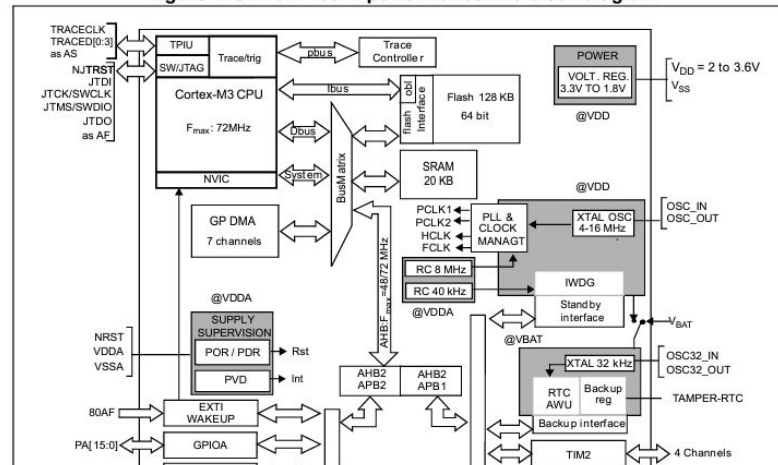
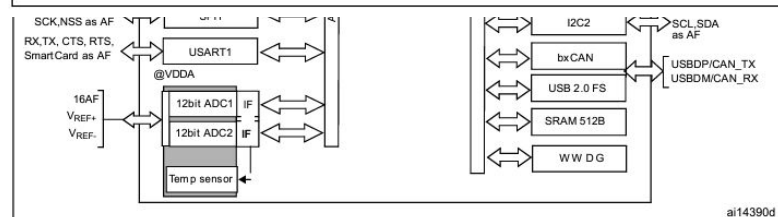
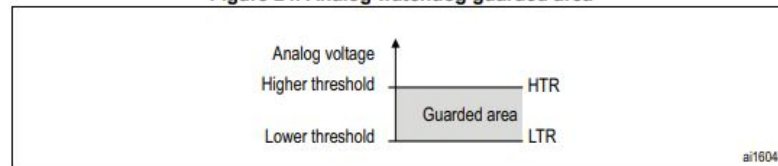


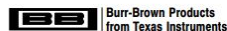
Figure 24. Analog watchdog guarded area



12.1 DAC introduction



The DAC module is a 12-bit, voltage output digital-to-analog converter. The DAC can be configured in 8- or 12-bit mode and may be used in conjunction with the DMA controller. In 12-bit mode, the data could be left- or right-aligned. The DAC has two output channels, each with its own converter. In dual DAC channel mode, conversions could be done independently or simultaneously when both channels are grouped together for synchronous update operation. An input reference pin V_{REF+} (shared with ADC) is available for better resolution.



DAC8814

SBAS3380 - JANUARY 2005 - REVISED SEPTEMBER 2006

Quad, Serial Input 16-Bit Multiplying Digital-to-Analog Converter

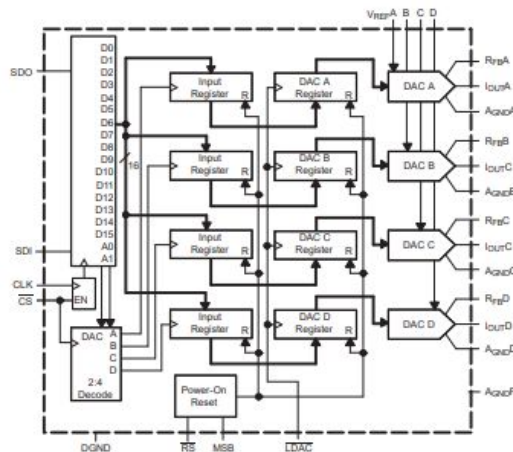
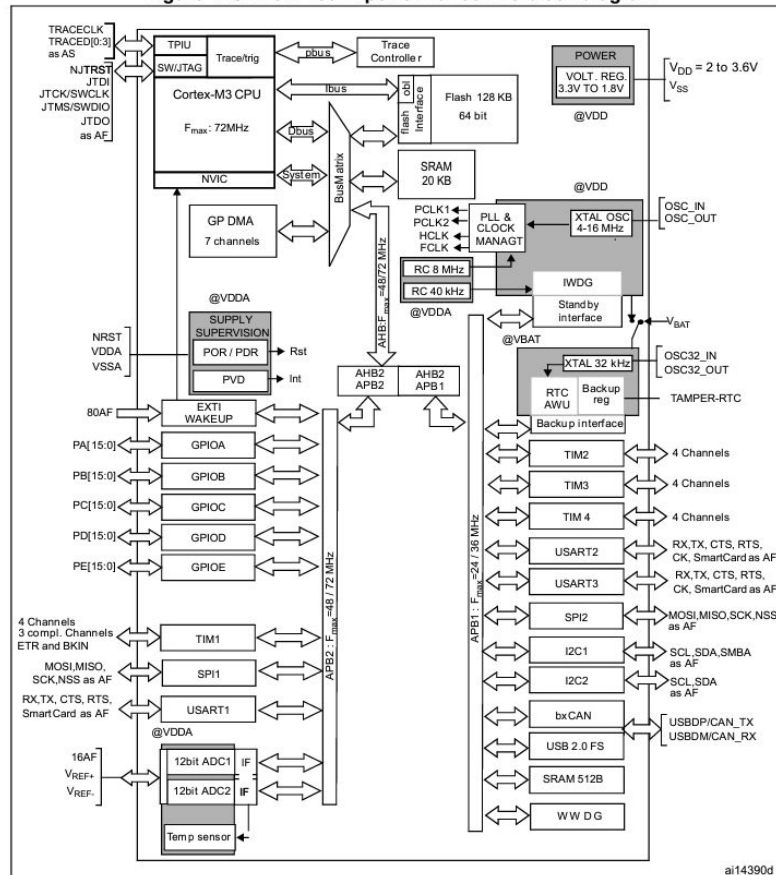


Figure 1. STM32F103xx performance line block diagram



18.1 RTC introduction

The real-time clock is an independent timer. The RTC provides a set of continuously running counters which can be used, with suitable software, to provide a clock-calendar function. The counter values can be written to set the current time/date of the system.

The RTC core and clock configuration (RCC_BDCR register) are in the Backup domain, which means that RTC setting and time are kept after reset or wakeup from Standby mode.

After reset, access to the Backup registers and RTC is disabled and the Backup domain (BKP) is protected against possible parasitic write access. To enable access to the Backup registers and the RTC, proceed as follows:

- enable the power and backup interface clocks by setting the PWREN and BKPEN bits in the RCC_APB1ENR register
- set the DBP bit the Power Control register (PWR_CR) to enable access to the Backup registers and RTC.

Figure 179. RTC simplified block diagram

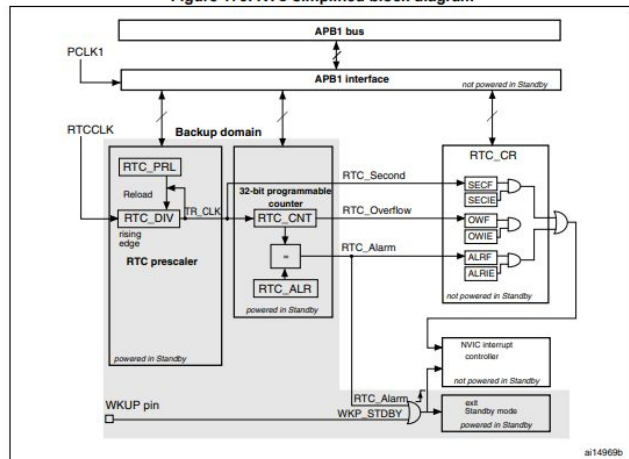
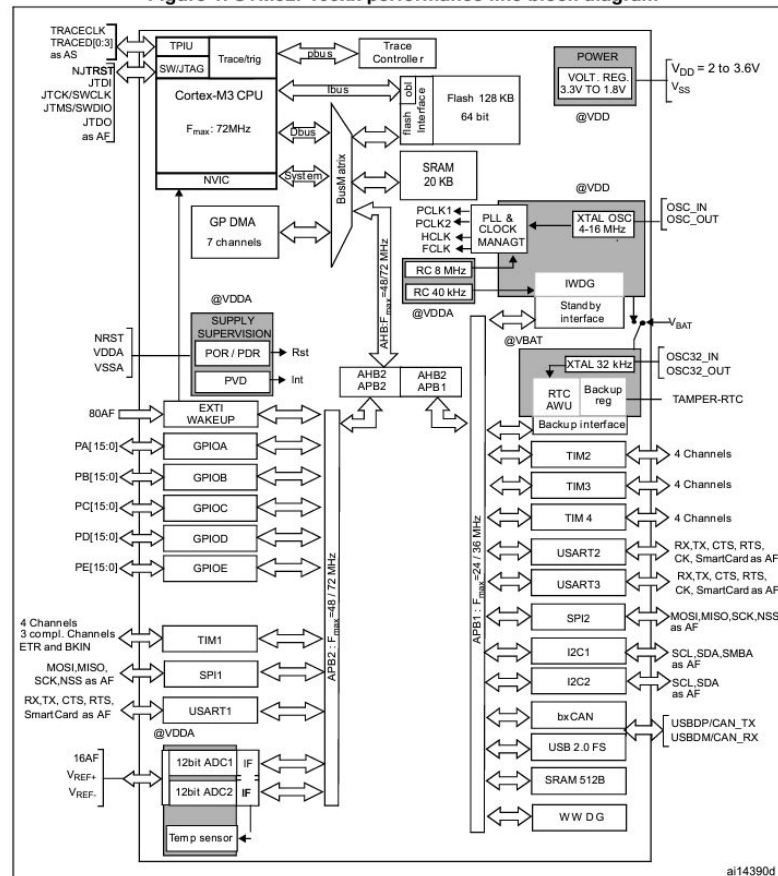


Figure 1. STM32F103xx performance line block diagram



4.1 CRC introduction

The CRC (cyclic redundancy check) calculation unit is used to get a CRC code from a 32-bit data word and a fixed generator polynomial.

Among other applications, CRC-based techniques are used to verify data transmission or storage integrity. In the scope of the EN/IEC 60335-1 standard, they offer a means of verifying the Flash memory integrity. The CRC calculation unit helps compute a signature of the software during runtime, to be compared with a reference signature generated at link-time and stored at a given memory location.

4.2 CRC main features

- Uses CRC-32 (Ethernet) polynomial: $0x4C11DB7$
 $- X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + X + 1$
- Single input/output 32-bit data register
- CRC computation done in 4 AHB clock cycles (HCLK)
- General-purpose 8-bit register (can be used for temporary storage)

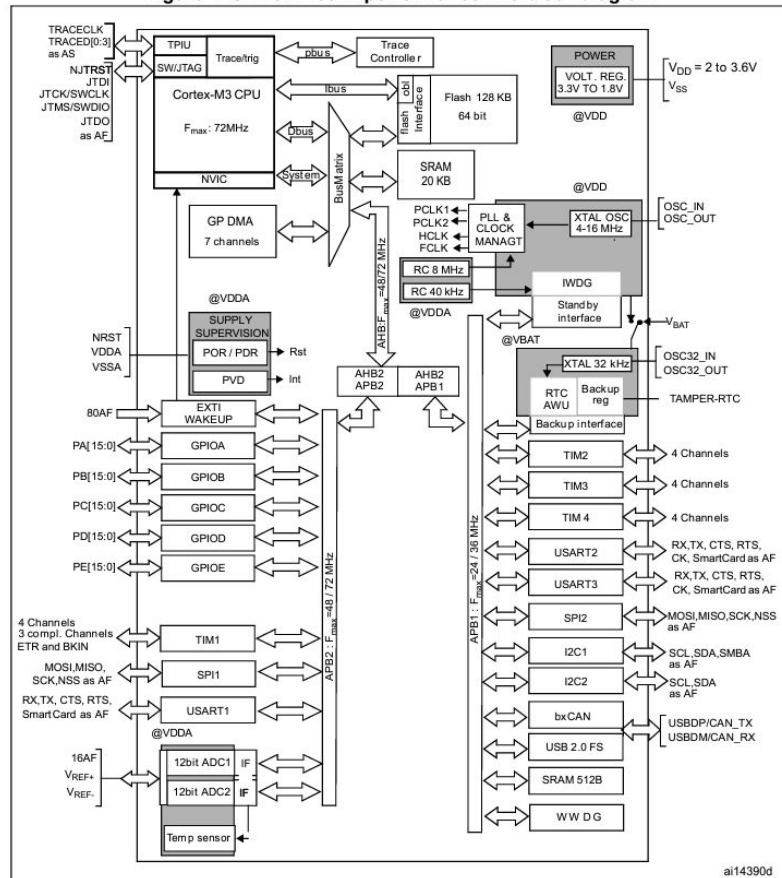
The block diagram is shown in [Figure 3](#).



AN3307 Application note

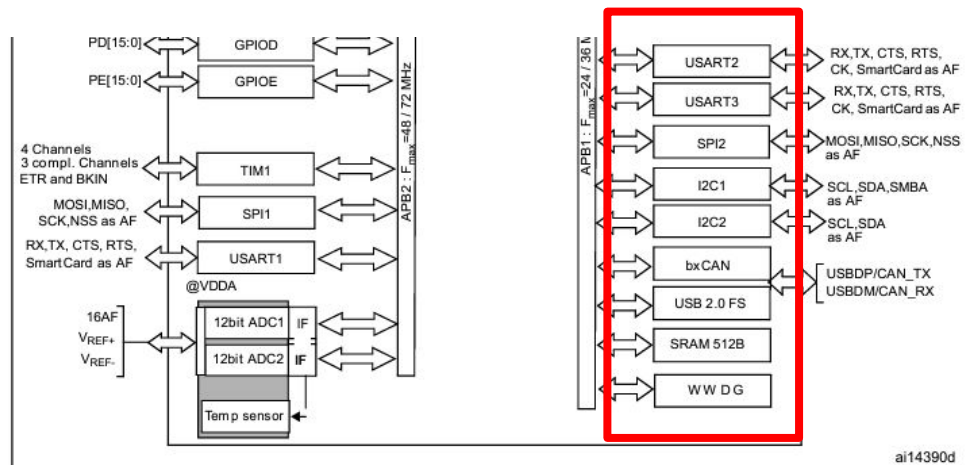
Guidelines for obtaining IEC 60335
Class B certification for any STM32 application

Figure 1. STM32F103xx performance line block diagram



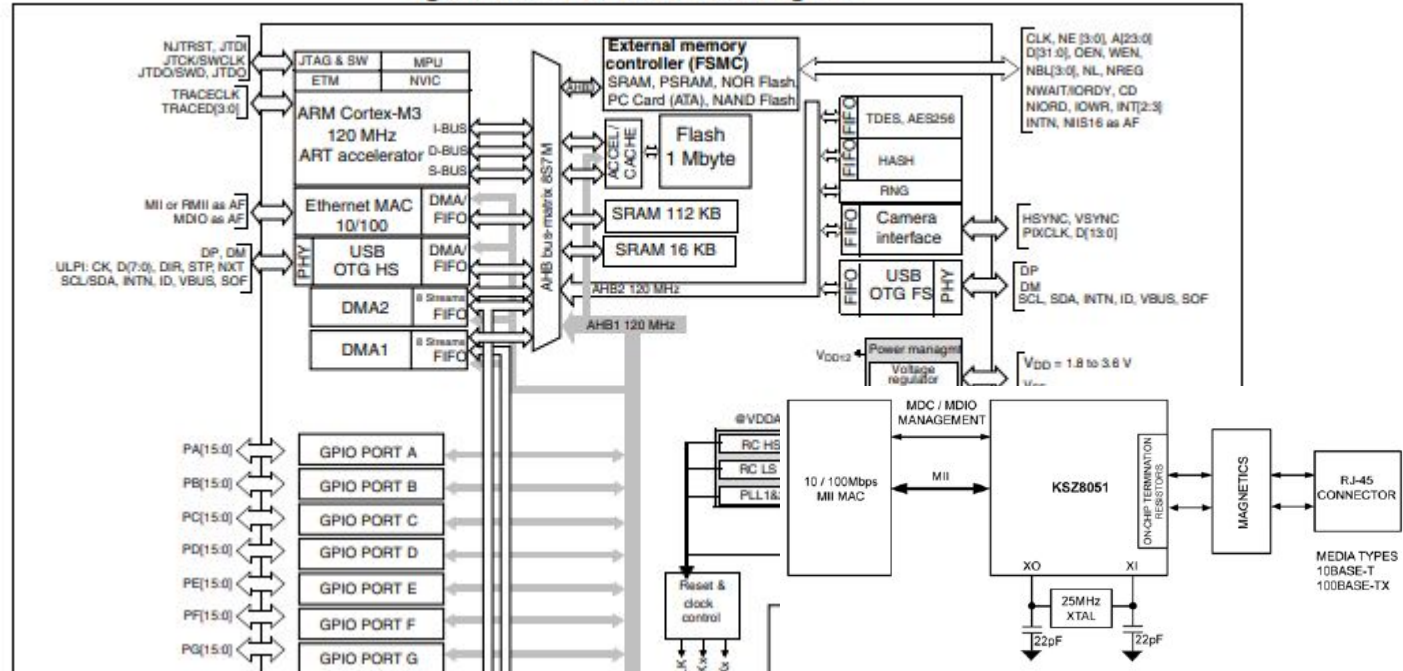
Comunicación

- Protocolos de capa física y capa de enlace
 - Requieren pocas acciones por parte del procesador
 - Sus requisitos de tiempo real son limitados ya que se encarga el hardware
 - Usando DMA para las transferencias, la carga del procesador es muy baja
- Protocolos de stack completo
 - Protocolos más complejos como el USB requieren librerías de software corriendo
 - Tienen requisitos de tiempo real exigentes (por ejemplo, responder a mensajes periódicamente para no ser expulsados del bus)
 - En uC sencillos consumen recursos importantes: la tendencia actual es tener un coprocesador o descargar esas tareas en otro núcleo
 - En algunos casos, un periférico puede absorber parte de la carga



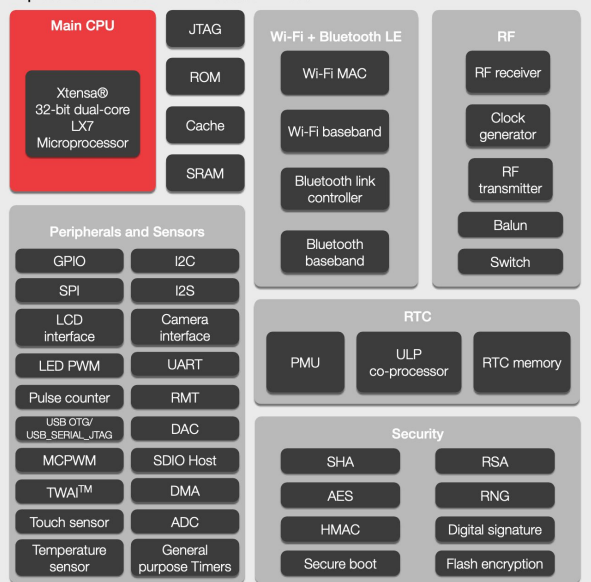
MCUs con más periféricos

Figure 4. STM32F21x block diagram



Miles de periféricos

Espressif's ESP32-S3 Wi-Fi + Bluetooth LE SoC



TEXAS INSTRUMENTS

Enhanced Pulse Width Modulation (ePWM)

The C2000 high-performance control peripherals are an integral component for all digital control systems. This module will cover pulse width modulation (PWM).

TEXAS INSTRUMENTS

Enhanced Capture Module (eCAP)

The C2000 enhanced capture module allows time-based logging of external signal transitions. It is used to accurately time external events by time stamping transitions on the capture input pin.

NXP ARM Cortex-M MCUs Powerhouse Portfolio

	ATTRIBUTES	PORTFOLIO				TARGET APPLICATIONS
		M0/M0+	M3/M4		M7	
General Purpose	Performance efficiency		LPC5410x, M4 w/M0+ Copr. 100MHz Power Efficiency up to 512K Flash, 104K SRAM	LPC5411x, M4 w/M0+ Copr. Flex Comm Interfaces, Voice Triggering X-less USB, up to 256K Flash, 192K SRAM		Always-on devices Voice control IoT
	Advanced integration	KL80/81, M0+ HW Security Crypto, USB, FlexIO up to 128K Flash, 96K SRAM	K64/66/65, M4 ENET, USB FS/HS, Security (KG5) up to 2MB Flash, 256K SRAM	K80/81, M4 HW Security Crypto, Tamper, CSPI, USB up to 256K Flash, 256K SRAM		Industrial control Payment, Metering Wearables, Display UI Printers
	Mainstream	LPC1100, M0+/M0 Options CAN or USB, EEPROM up to 256K Flash, 36K SRAM	KL28, M0+ 72/96MHz, FlexIO, BootROM, more I/O up to 512K Flash, 128K SRAM	LPC4000, M4 Graphic LCD, FS USB, ENET, CAN up to 512K Flash, 96K SRAM	K22/24/26, M4 Large memory, x-less USB up to 2M Flash, 256K RAM	Consumer/Gaming Accessories Wearables Home Automation Data Concentrators Industrial
	Low-cost & Small Form Factor	LPC117, M0+ FlexIO, BootROM Up to 256K Flash, 32K RAM	KL27, M0+ FlexIO, BootROM, x-less USB Up to 256K Flash, 32K RAM	LPC1700, M3 Graphics LCD, FS USB, ENET, CAN up to 512K Flash, 96K SRAM	LPC1500, M3 CAN, Advanced Analog, PWMs up to 256K Flash, 36K RAM	Wake-up unit System task/Co-processor Power Management Control System
Application Specific	Wireless Connectivity	LPC81x/82x, M0+ 30MHz, ADC, Low Pin Count up to 32K Flash, 8K RAM	KL05, M0+ Low Power, DAC, TSI 8-32K Flash, 1-4K RAM	KL02/03, M0+ Small Form Factor, Low Power 8-32K Flash, 1-4K RAM		Home Automation Thread BLE Apple HomeKit
	Motor Control & Power Conversion	KW20/30/40Z, M0+ 802.15.4+BLE 4.1 Radio, DC/DC 160K Flash, 20K RAM	KW21/31/41Z, M0+ 802.15.4+BLE 4.2 Radio, DC/DC, Balun Up to 512K Flash, Up to 128K RAM	KW2xD, M4 802.15.4 Radio, Thread Up to 512K Flash & 64K RAM		BLDC / PMSM motors Photo voltaic Industrial Controls
	5V Robust	KW01, M0+ Sub-GHz 512K Flash, 128K RAM	KV1x, M0+ BLDC, entry-level PMSM Up to 128K Flash, Up to 16K RAM	KV3x, M4 Mid-range PMSM, UPS power control, KMS Up to 512K Flash, Up to 96K RAM	KV4x, M4 High-perf motors, UPS, solar & mid-range AC/DC control, KMS Up to 256K Flash, Up to 32K RAM	Appliance Smart Lighting
		KE02/4/6, M0+ ADC, CAN (KE06) Up to 128K Flash, Up to 16K RAM, 256K EEPROM	KE1xF (M4F) / KE1xZ (M0+) Higher perf. & more comm. interfaces 256K / 512K Flash, 32K / 64K RAM, 2xCAN, new TSI (2)			

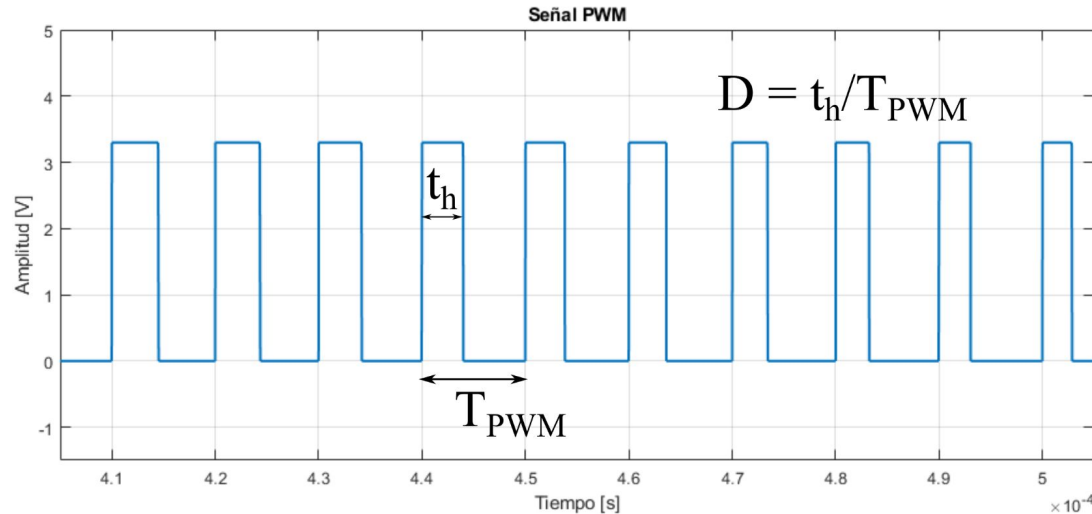
(not a complete portfolio summary)

KMS = Kinetis Motor Suite

PWM

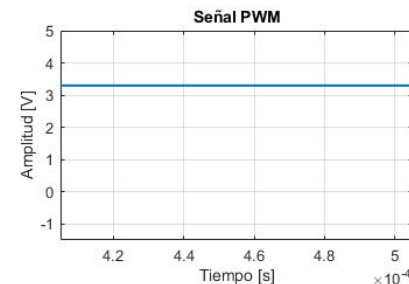
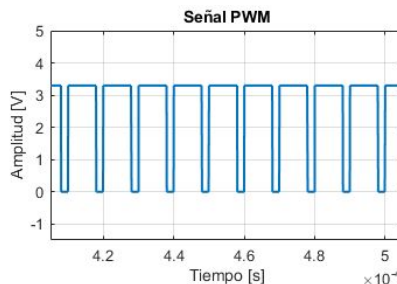
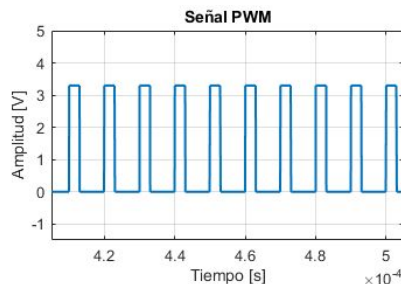
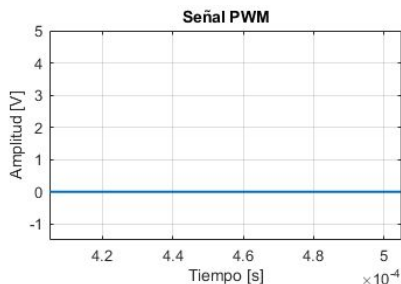
Modulación por ancho de pulso

- El PWM permite codificar la información en el tiempo en lugar de la amplitud o de un código binario particular.
- El ciclo de trabajo, duty cycle, D codifica el valor.

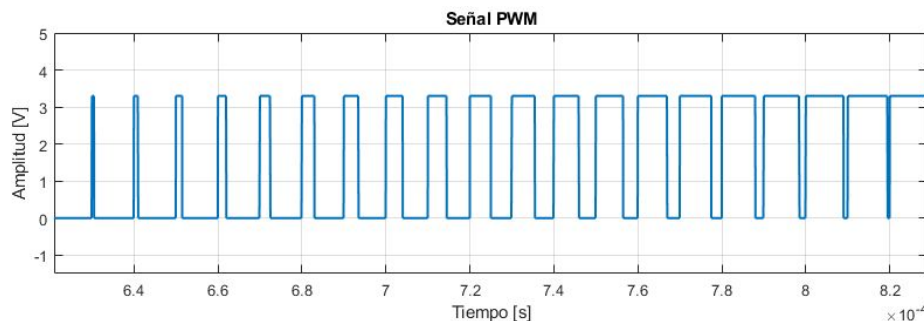


Ancho de pulso o “duty cycle”

- Ejemplos para 0%, 30%, 80%, 100%

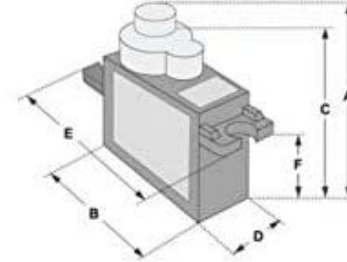


- Rampa con incrementos de 5%



Usos del PWM

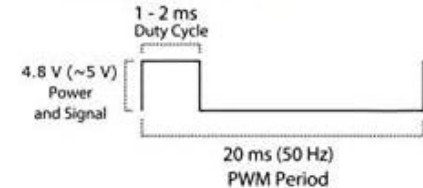
- Un uso es la codificación directa del valor en tiempo
 - e.g. servomecanismos reciben un valor de 0 a 255 y lo mapean en un ángulo de 0 a 180°
- Otro uso es variar la potencia promedio entregada a un motor, luz, etc
- El otro uso es un DAC de bajo costo
 - Una forma intuitiva de pensarlo es que el promedio de la señal de PWM en un período devuelve el valor codificado.
 - El PWM produce una modulación no lineal de la señal codificada. Si está acotada en frecuencia, puede recuperarse con un filtro pasa bajos.
 - Cuanta más alta la frecuencia del PWM más fácil será recuperar la onda.



Position "0°" (1.5 ms pulse) is middle, "90°" (~2ms pulse) is middle, is all the way to the right, "-90°" (~1ms pulse) is all the way to the left.

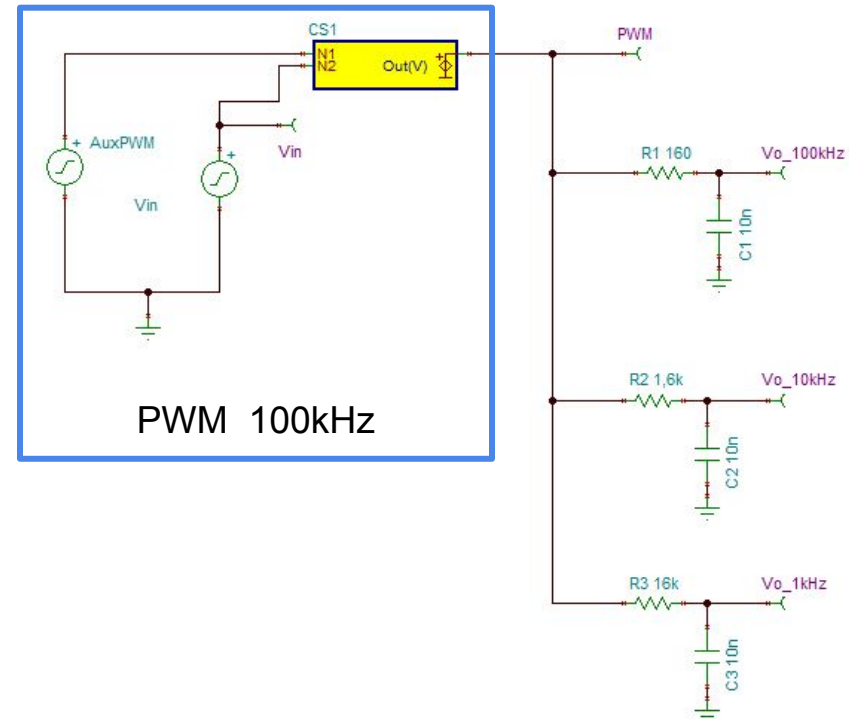
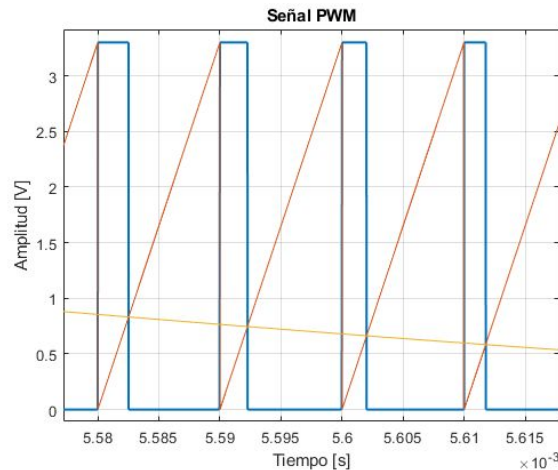
Dimensions & Specifications	
A (mm) :	32
B (mm) :	23
C (mm) :	28.5
D (mm) :	12
E (mm) :	32
F (mm) :	19.5
Speed (sec) :	0.1
Torque (kg-cm) :	2.5
Weight (g) :	14.7
Voltage :	4.8 - 6

PWM=Orange (⏏)
Vcc=Red (+)
Ground=Brown (-)

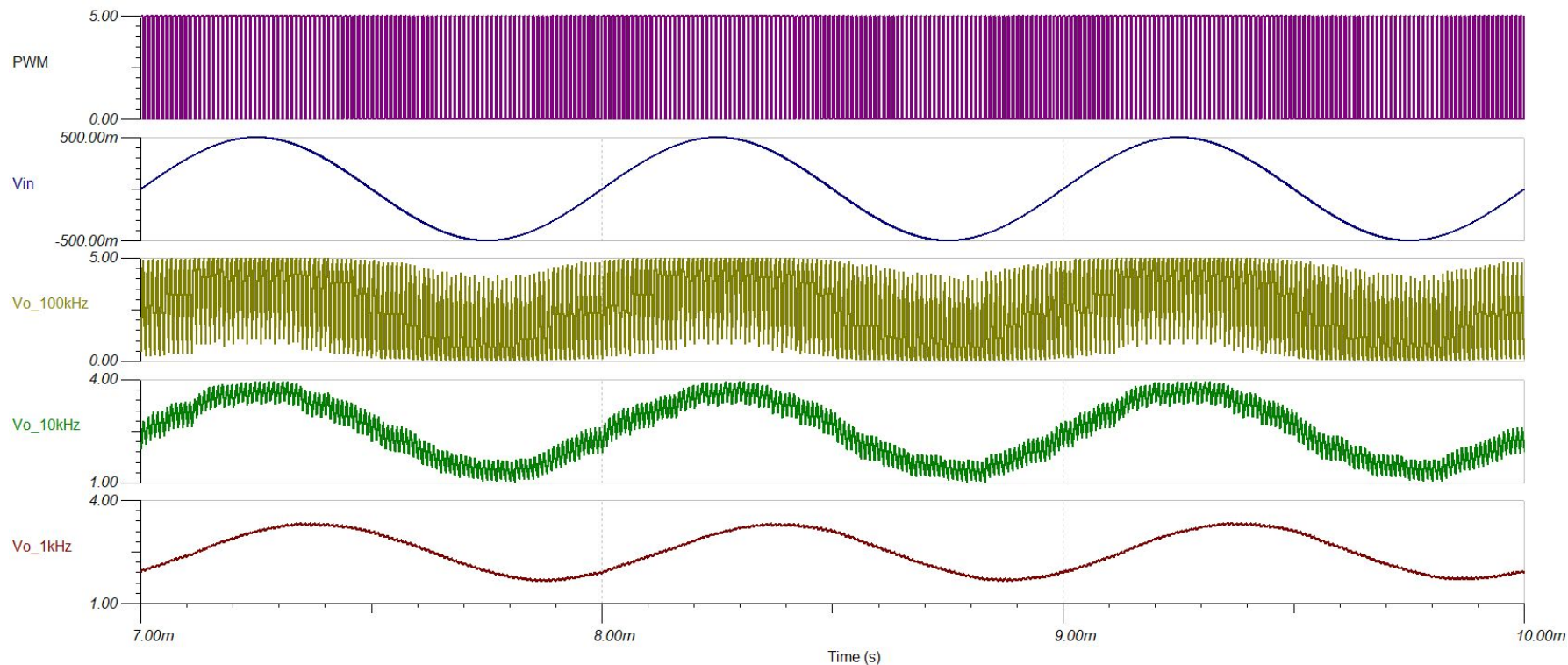


Ejemplo simulado de generación y recuperación de señal

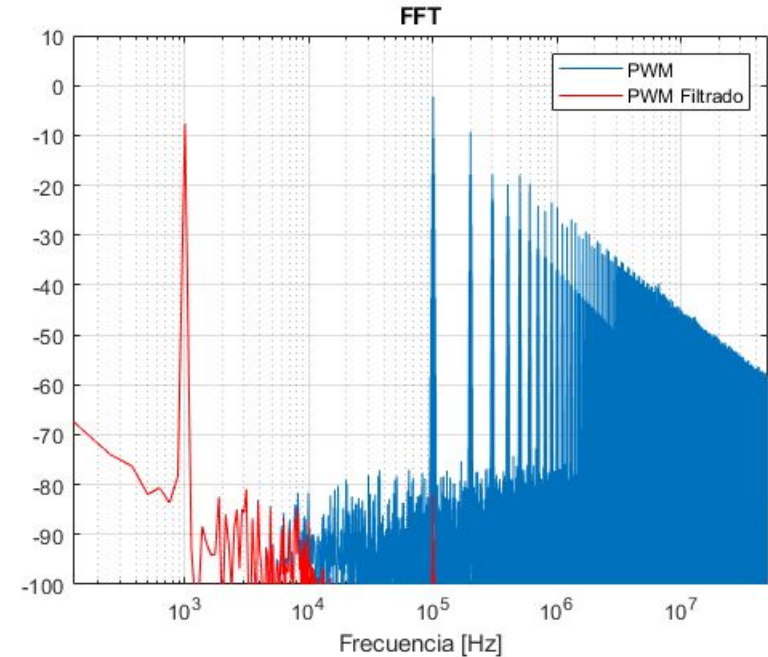
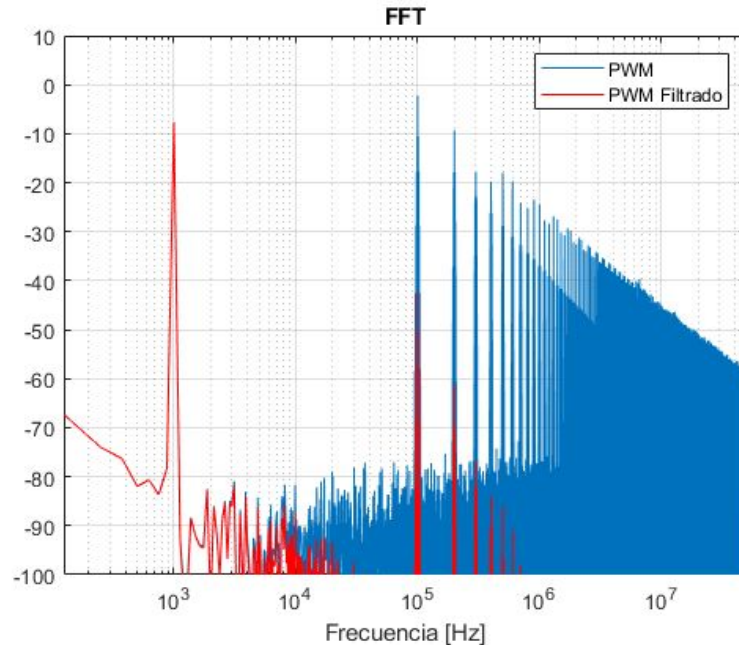
- Simulación en Spice Tina de TI
- Generamos el PWM con un diente de sierra DS y un seno SEN
- Siempre que $SEN > DS$ la señal será alta, si no, será 0



Recuperación de señal



Efecto del filtrado en onda sinusoidal

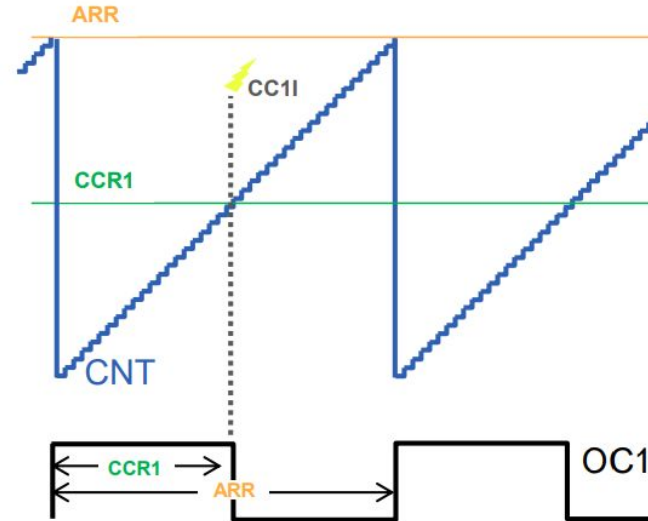


PWM 100 kHz, señal codificada seno 1 kHz

Filtro pasa bajos butterworth $f_c = 10$ kHz, 1 polo vs 2 polos

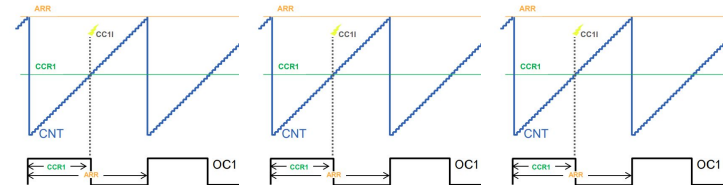
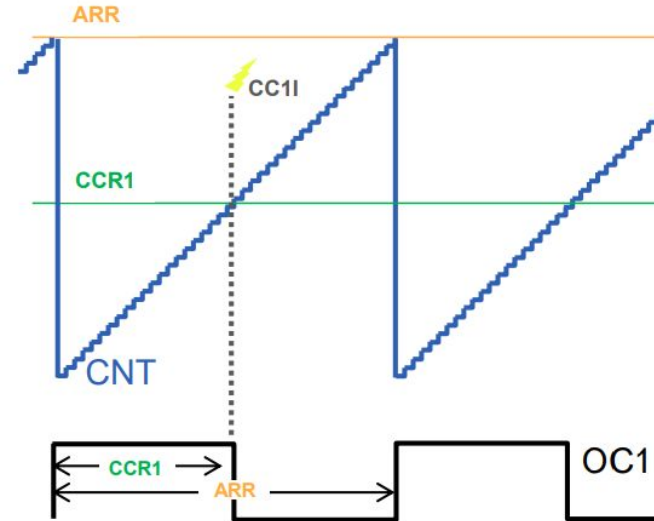
Implementación del PWM en microcontroladores

- El PWM suele estar asociado a los timers y es un modo de operación de los mismos
- Supongamos un timer que cuenta ascendentemente en su registro CNT desde 0 y se resetea cada vez que alcanza un valor ARR
- Es esencialmente una onda diente de sierra
- Se produce una salida positiva siempre que un valor $CCR1 > CNT$ y 0 en el caso contrario
- Si actualizamos el valor de CCR1 ciclo a ciclo estaremos codificando las muestras de una señal



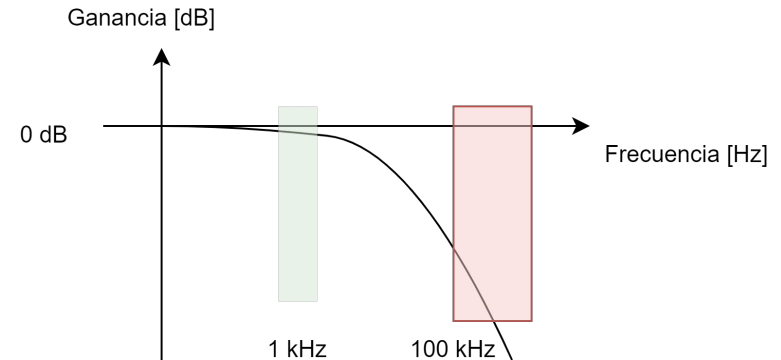
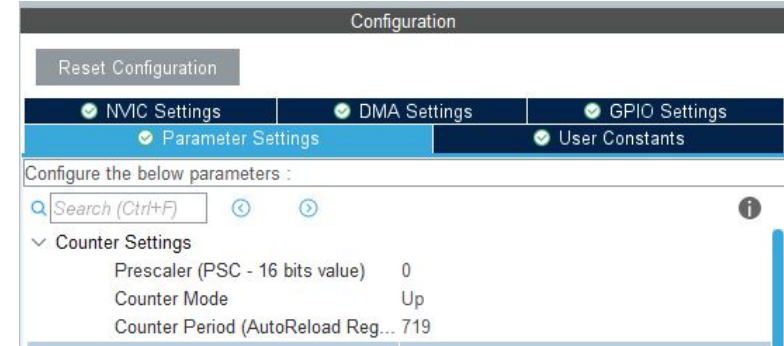
Preguntas:

- ¿Cuál será la resolución de nuestro “DAC” basado en el PWM?
- ¿Qué relación habrá entre la frecuencia del timer y la resolución del PWM? (para hacerlo más fácil, pensemos que queremos lograr la máxima frecuencia posible por “muestra”)

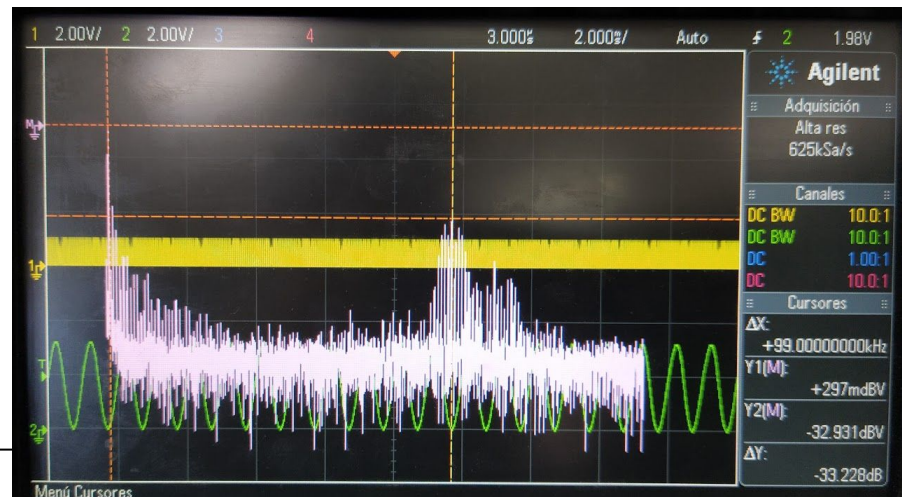
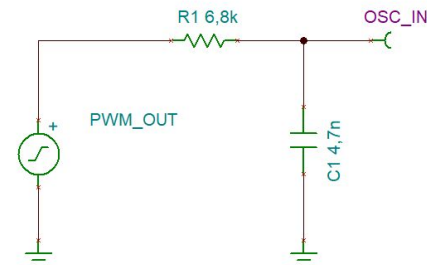


Ejemplo de PWM en el STM32F103C8 - Sinusoide

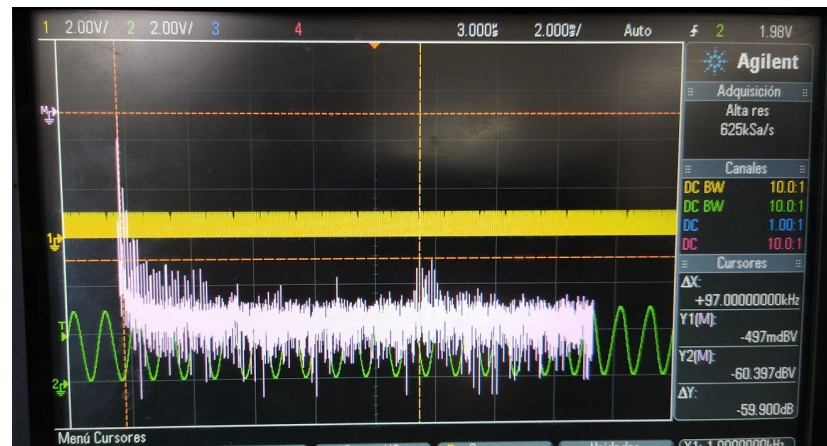
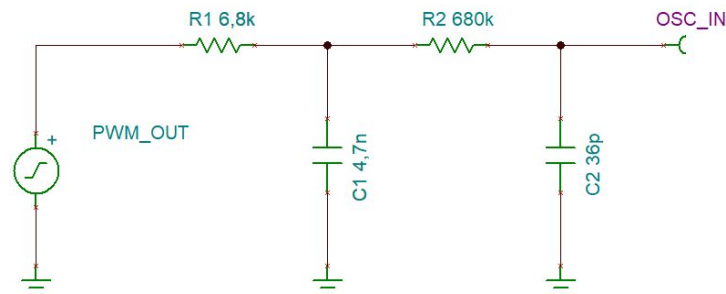
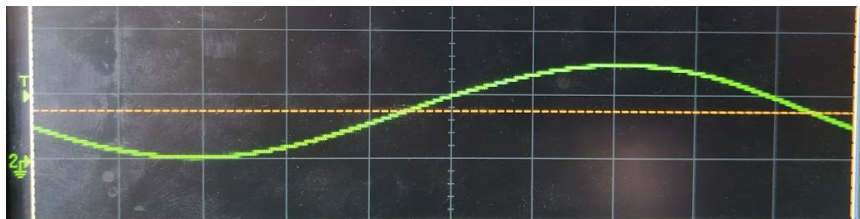
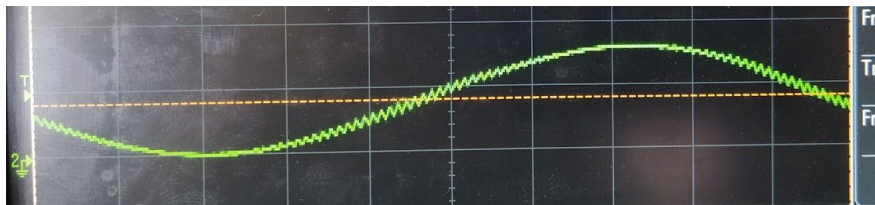
```
#include "main.h"
#include <math.h>
#define NSEN 100
#define VMAX 720
uint16_t onda[NSEN] = {0};
int idx=0;
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_TIM2_Init();
    /* USER CODE BEGIN 2 */
    for(idx = 0; idx < NSEN ; idx++){
        onda[idx] = (uint16_t)((float)VMAX/2.0*(sin(2.0*M_PI*idx/(float)NSEN)+1.0));
    }
    HAL_TIM_PWM_Start_IT(&htim2, TIM_CHANNEL_1);
    /* USER CODE END 2 */
    while (1) { }
}
/* USER CODE BEGIN 4 */
void HAL_TIM_OC_DelayElapsedCallback(TIM_HandleTypeDef *htim){
    GPIOC->BSRR |= GPIO_BSRR_BS13_Msk;
    idx = (idx+1)%NSEN;
    TIM2->CCR1 = onda[idx];
    GPIOC->BSRR |= GPIO_BSRR_BR13_Msk;
}
/* USER CODE END 4 */
```



Ejemplo de PWM en el STM32F103C8 - Sinusoide

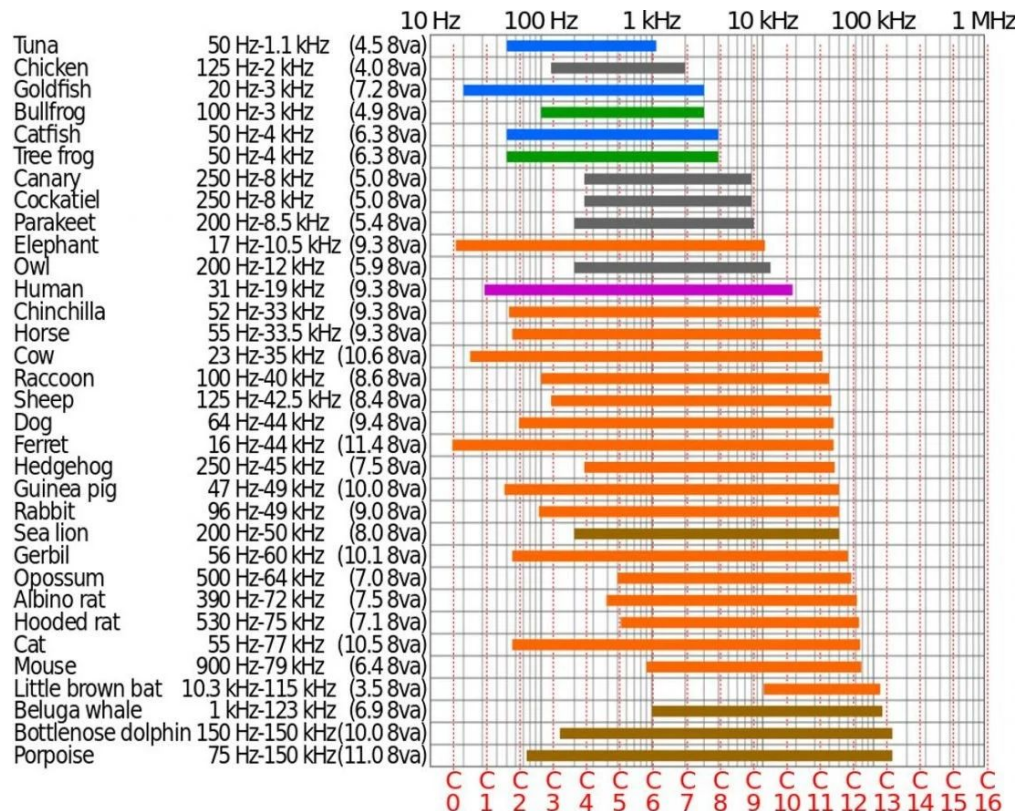


Ejemplo de PWM en el STM32F103C8 - Sinusoide



Audio y PWM

- Un seno sería un “tono”, pero podemos codificar cualquier señal con PWM, entre ellas señales de audio
- Debemos considerar el ancho de banda de la señal
- El teléfono analógico tenía un ancho de banda de 300 a 3400 Hz



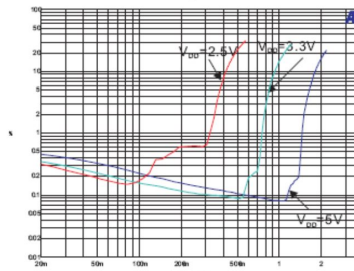
Amplificador PAM8403

Recommended Operating Conditions (@T_A = +25°C, unless otherwise specified.)

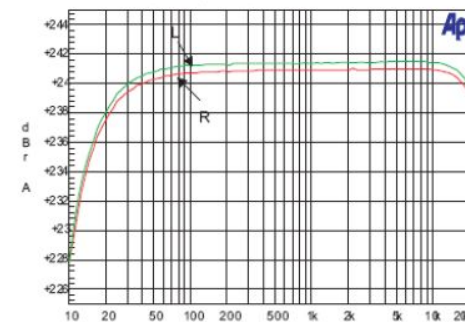
Parameter	Rating	Unit
Supply Voltage Range	2.5 to 5.5	V
Operation Temperature Range	-40 to +85	°C
Junction Temperature Range	-40 to +125	°C

G _V	Closed Loop Gain	V _{DD} = 3.3V, P _O = 0.2W, R _L = 4Ω		U.11	
		V _{DD} = 3V to 5V		24	dB
PSRR	Power Supply Ripple Rejection	V _{DD} = 5.0V, Inputs AC-Grounded with C _{IN} = 0.47μF	f = 100Hz	-59	dB
			f = 1kHz	-58	dB
C _S	Crosstalk	V _{DD} = 5.0V, P _O = 0.5W, R _L = 8Ω, G _V = 20db	f = 1kHz	-95	dB
SNR	Signal-to-Noise Ratio	V _{DD} = 5.0V, V _{ORMS} = 1V, G _V = 20db	f = 1kHz	80	dB
V _N	Output Noise	V _{DD} = 5.0V, Inputs AC-Grounded with C _{IN} = 0.47μF	No A-Weighting	100	μV
			A-Weighting	150	μV
Dyn	Dynamic Range	V _{DD} = 5.0V, THD = 1%	f = 1kHz	90	dB

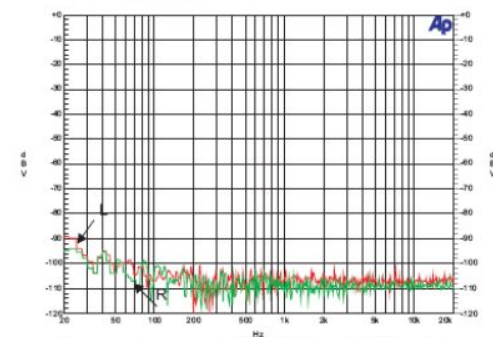
2. THD+N vs Output Power

R_L = 8Ω, Gain = 24dB, f = 1kHz

5. Frequency response

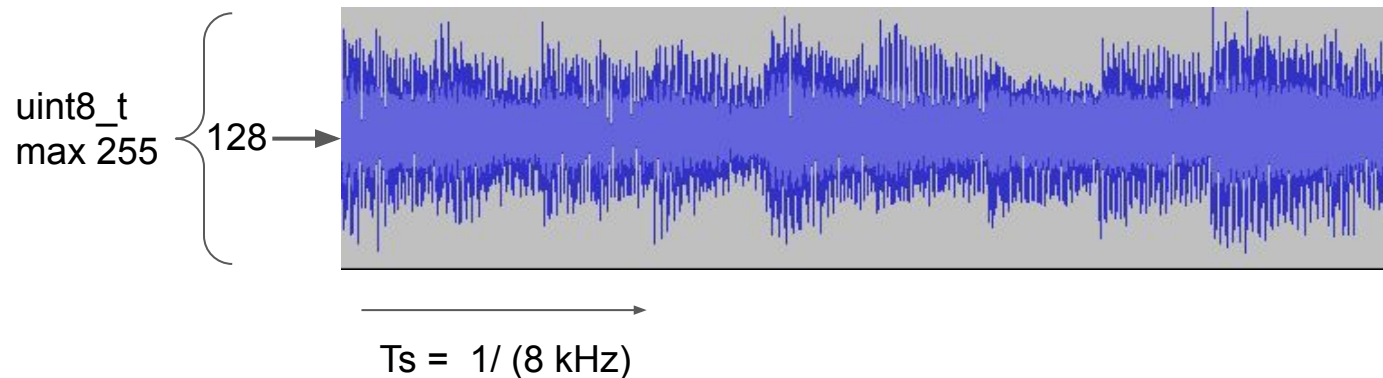
V_{DD} = 5V, R_L = 8Ω, Gain = 24dB, C_{in} = 1μF

7. Noise Floor FFT

Inputs ac-ground, V_{DD} = 5V, R_L = 8Ω, C_{in} = 1μF

Ejemplo - Reproducción de música - WAV

- Los archivos que codifican música se graban a una tasa, codificación y compresión según el formato.
- Un tipo agradablemente primitivo es el .wav que registra directamente la forma de onda sin compresión en valores sin signo, con media igual a la mitad del rango



Ejemplo - Reproducción de música - WAV

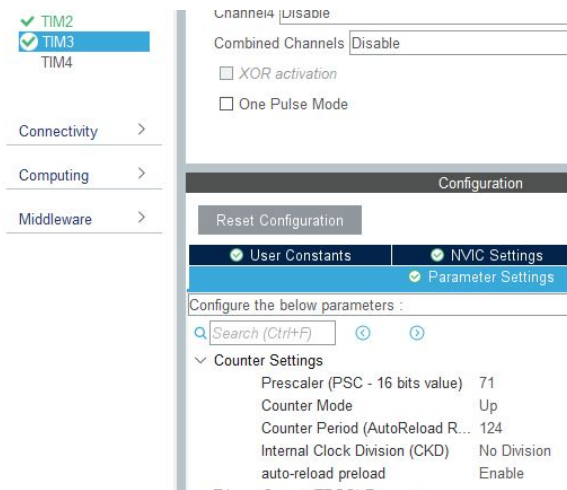
- Cualquier .wav puede formatearse en un programa de edición de audio (audacity, goldwave) y luego manipularse para un programa de C con un editor como HxD (<https://mh-nexus.de/en/hxd/>)

```
main.c  musica.h X
1  /*
2   * musica.h
3   *
4   * Created on: Oct 24, 2022
5   * Author: Fede
6   */
7
8  #ifndef INC_MUSICA_H_
9  #define INC_MUSICA_H_
10
11  #define NMUSICA 48772
12
13  const uint8_t musica[NMUSICA] = {
14      0x52, 0x49, 0x46, 0x46, 0x7C, 0xBE, 0x00, 0x00, 0x57, 0x41, 0x56, 0x45,
15      0x66, 0x6D, 0x74, 0x20, 0x10, 0x00, 0x00, 0x00, 0x01, 0x00, 0x01, 0x00,
16      0x40, 0x1F, 0x00, 0x00, 0x40, 0x1F, 0x00, 0x00, 0x01, 0x00, 0x08, 0x00,
17      0x64, 0x61, 0x74, 0x61, 0x58, 0xBE, 0x00, 0x00, 0x7F, 0x80, 0x80, 0x7F,
18      0x80, 0x80, 0x80, 0x80, 0x7F, 0x80, 0x7F, 0x80, 0x7F, 0x80, 0x7F,
19      0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x7F, 0x7F, 0x7F, 0x80,
20      0x80, 0x7F, 0x80, 0x80, 0x7F, 0x7F, 0x80, 0x7F, 0x80, 0x80, 0x80, 0x7F,
21      0x80, 0x80, 0x7F, 0x7F, 0x80, 0x80, 0x7F, 0x80, 0x80, 0x7F, 0x7F, 0x80,
22      0x7F, 0x7F, 0x80, 0x80, 0x7F, 0x7F, 0x80, 0x80, 0x7F, 0x7F, 0x80, 0x7F,
```

Ejemplo - Reproducción de música

- Como vimos, cuanto más rápido el PWM, más fácil es filtrar la señal para recuperarla en banda base
- Una estrategia puede ser poner el PWM a funcionar lo más rápido posible y establecer la temporización de la reproducción con otro timer

El timer 2 quedó afectado a la generación de PWM



El 3 se configura para una tasa de actualización de 8 kHz

Con interrupciones habilitadas

Ejemplo - Reproducción de música

```

1  /*
2  * musica.h
3  *
4  * Created on: Oct 24, 2022
5  * Author: Fede
6  */
7
8  #ifndef INC_MUSICA_H_
9  #define INC_MUSICA_H_
10
11 #define NMUSICA 48772
12
13 const uint8_t musica[NMUSICA] = {
14     0x52, 0x49, 0x46, 0x46, 0x7C, 0xBE, 0x00, 0x00, 0x57, 0x41, 0x56, 0x45,
15     0x66, 0x6D, 0x74, 0x20, 0x10, 0x00, 0x00, 0x00, 0x01, 0x00, 0x01, 0x00,
16     0x40, 0x1F, 0x00, 0x00, 0x40, 0x1F, 0x00, 0x00, 0x01, 0x00, 0x08, 0x00,
17     0x64, 0x61, 0x74, 0x61, 0x58, 0xBE, 0x00, 0x00, 0x7F, 0x80, 0x80, 0x7F,
18     0x80, 0x80, 0x80, 0x80, 0x7F, 0x80, 0x7F, 0x80, 0x80, 0x7F, 0x80, 0x7F,
19     0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x7F, 0x7F, 0x7F, 0x80,
20     0x80, 0x7F, 0x80, 0x80, 0x7F, 0x7F, 0x80, 0x7F, 0x80, 0x80, 0x80, 0x7F,
21     0x80, 0x80, 0x7F, 0x7F, 0x80, 0x80, 0x7F, 0x80, 0x80, 0x7F, 0x7F, 0x80,
22     0x7F, 0x7F, 0x80, 0x80, 0x7F, 0x7F, 0x80, 0x80, 0x7F, 0x7F, 0x80, 0x7F,

```

```

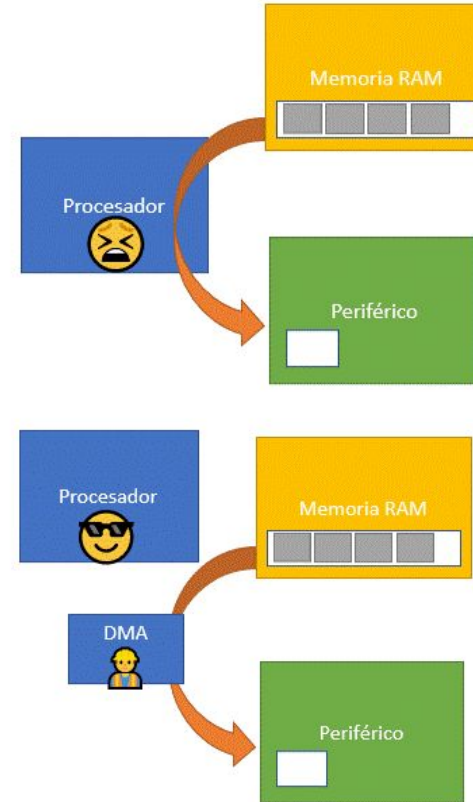
#include "main.h"
#include "musica.h"
TIM_HandleTypeDef htim2;
TIM_HandleTypeDef htim3;
int idx=0;
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_TIM2_Init();
    MX_TIM3_Init();
    /* USER CODE BEGIN 2 */
    HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
    HAL_TIM_Base_Start_IT(&htim3);
    /* USER CODE END 2 */
    while (1) { }
}
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef htim) {
    GPIOC->BSRR |= GPIO_BSRR_BS13_Msk;
    idx = (idx + 1) % NMUSICA;
    TIM2->CCR1 = musica[idx];
    GPIOC->BSRR |= GPIO_BSRR_BR13_Msk;
}
/* USER CODE END 4 */

```

DMA

Direct Memory Access

- El DMA es un periférico que se conecta al bus de datos y transfiere, autónomamente, datos desde una fuente a un destino.
- En el STM32F103 tanto la fuente como el destino pueden ambos ser periféricos o memoria. En general en los MCUs algunos periféricos están conectados al DMA y otros no.
- El DMA es muy útil porque libera al procesador del tiempo que ocupan las transferencias: es “como” tener un segundo procesador corriendo en paralelo
- Por supuesto, si hay conflictos deben arbitrarse y el procesador puede perder tiempo intentando acceder a una memoria o periférico ocupada por el controlador de DMA



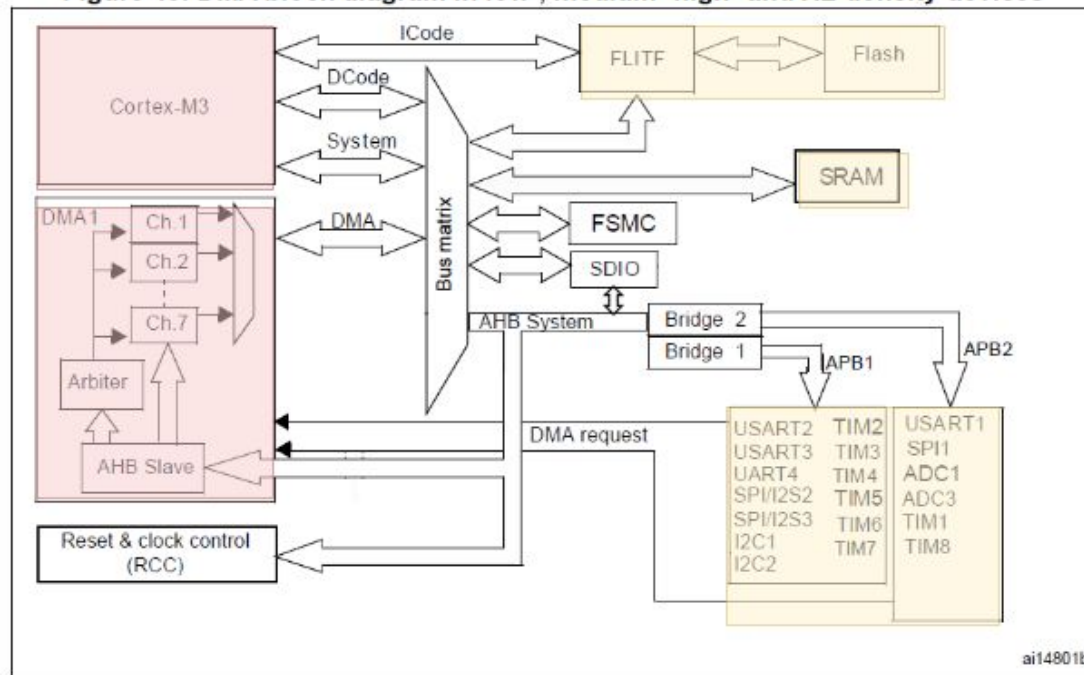
Algunos casos-ejemplo de uso

- El MCU se conecta a un ADC por SPI. Cada vez que hay una muestra disponible se dispara una interrupción en el MCU, y comienza una transferencia SPI asistida por DMA.
- Inversamente, el MCU se conecta a un DAC por SPI y se envían las muestras de memoria automáticamente.
- El MCU procesa un arreglo de datos (e.g. realizando un filtrado) y deben trasladarse los datos de un buffer en memoria a otro.

DMA en el STM32F103C8 - Conexionado

- El controlador DMA1 tiene dos puertos que actúan como **maestros** en el Bus AHB y uno **esclavo**
 - El **Puerto de Memoria**: Puede comunicarse con la memoria flash o la sram.
 - El **puerto de periféricos**: Puede comunicarse con algún subset de periféricos y memoria (según el canal)
 - El puerto esclavo para ser configurado por el procesador
- El controlador tiene **7 canales**, cada uno opera independientemente y puede realizar transferencias entre los puertos en cualquier sentido
- Cada canal tiene una **Request Line** que al activarse dispara una o una secuencia de transferencias.
- Los canales tienen 4 niveles de prioridades programables que sirven para arbitrar conflictos en los pedidos de transferencias

Figure 49. DMA block diagram in low-, medium- high- and XL-density devices



DMA en el STM32F103C8 - Conexionado

- El arbitraje sólo tiene lugar cuando dos maestros necesitan acceder a la misma memoria esclava o periférico.
- Cuando existen conflictos se resuelven con un algoritmo de round robin modificado para asignarle un slice al CPU siempre

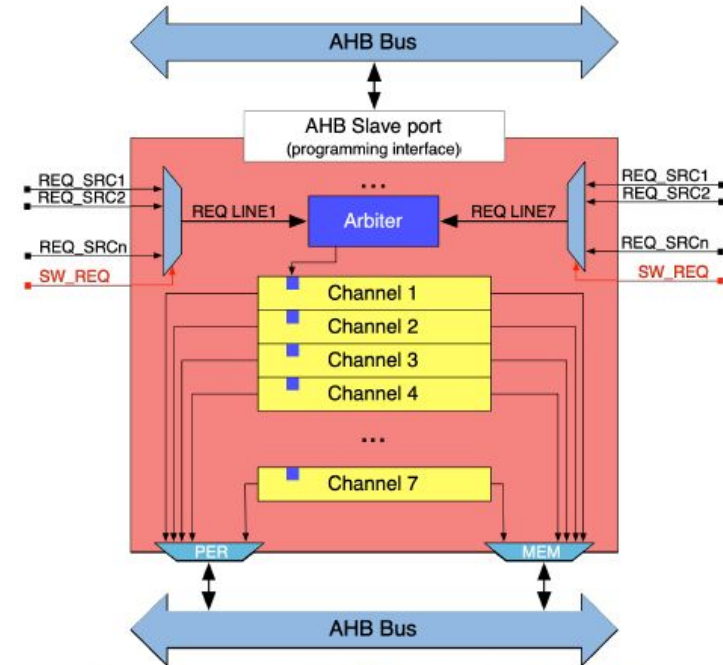


Figure 9.3: A representation of the DMA structure in F0/F1/F3/L0/L1/L4 MCUs

Maapeo de periféricos a canales DMA

Figure 50. DMA1 request mapping

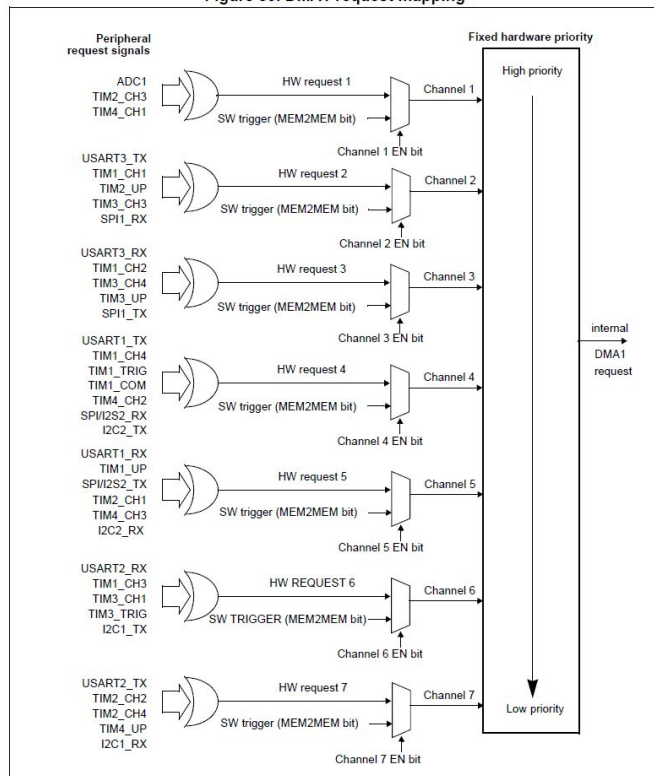


Table 78. Summary of DMA1 requests for each channel

Peripherals	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Channel 6	Channel 7
ADC1	ADC1	-	-	-	-	-	-
SPI/I ² S	-	SPI1_RX	SPI1_TX	SPI2/I2S2_RX	SPI2/I2S2_TX	-	-
USART	-	USART3_TX	USART3_RX	USART1_TX	USART1_RX	USART2_RX	USART2_TX
I ² C	-	-	-	I2C2_TX	I2C2_RX	I2C1_TX	I2C1_RX
TIM1	-	TIM1_CH1	-	TIM1_CH4 TIM1_TRIG TIM1_COM	TIM1_UP	TIM1_CH3	-
TIM2	TIM2_CH3	TIM2_UP	-	-	TIM2_CH1	-	TIM2_CH2 TIM2_CH4
TIM3	-	TIM3_CH3	TIM3_CH4 TIM3_UP	-	-	TIM3_CH1 TIM3_TRIG	-
TIM4	TIM4_CH1	-	-	TIM4_CH2	TIM4_CH3	-	TIM4_UP

DMA en el STM32F103C8 - Configuración de transferencias

- Tamaño de los datos: Ambos extremos pueden ser bytes, half-words o words **entremezclados**
- Para transferencias de varios datos puede configurarse que **cada puerto incremente su dirección, y un contador** (para cada canal) que se decrementa luego de cada transferencia hasta llegar a 0
- El controlador puede **disparar interrupciones** al completar $\frac{1}{2}$ transferencia o una transferencia completa
- El **modo circular** hace que se resetee el contador al llegar a 0 automáticamente

Channel configuration procedure

The following sequence should be followed to configure a DMA channelx (where x is the channel number).

1. Set the peripheral register address in the DMA_CPARx register. The data will be moved from/ to this address to/ from the memory after the peripheral event.
2. Set the memory address in the DMA_CMARx register. The data will be written to or read from this memory after the peripheral event.
3. Configure the total number of data to be transferred in the DMA_CNDTRx register. After each peripheral event, this value will be decremented.
4. Configure the channel priority using the PL[1:0] bits in the DMA_CCRx register
5. Configure data transfer direction, circular mode, peripheral & memory incremented mode, peripheral & memory data size, and interrupt after half and/or full transfer in the DMA_CCRx register
6. Activate the channel by setting the ENABLE bit in the DMA_CCRx register.

As soon as the channel is enabled, it can serve any DMA request from the peripheral connected on the channel.

Once half of the bytes are transferred, the half-transfer flag (HTIF) is set and an interrupt is generated if the Half-Transfer Interrupt Enable bit (HTIE) is set. At the end of the transfer, the Transfer Complete Flag (TCIF) is set and an interrupt is generated if the Transfer Complete Interrupt Enable bit (TCIE) is set.

Registros del DMA

Conf. general

- DMA interrupt status register (DMA_ISR)
 - GIFx: Global enable
 - TCIFx: Transfer complete
 - HTIFx: Half-transfer
 - TEIFx: Transfer error
- DMA interrupt flag clear register

Replicado para cada canal

Table 80. DMA register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
0x000	DMA_ISR	Reserved				TEIF7	HTIF7	TCIF7	GIF7	TEIF6	HTIF6	TCIF6	GIF6	TEIF5	HTIF5	TCIF5	GIF5	TEIF4	HTIF4	TCIF4	GIF4	TEIF3	HTIF3	TCIF3	GIF3	TEIF2	HTIF2	TCIF2	GIF2	TEIF1	HTIF1	TCIF1	GIF1					
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x004	DMA_IFCR	Reserved				CTEIF7	CHTIF7	CTCIF7	CGIF7	CTEIF6	CHTIF6	CTCIF6	CGIF6	CTEIF5	CHTIF5	CTCIF5	CGIF5	CTEIF4	CHTIF4	CTCIF4	CGIF4	CTEIF3	CHTIF3	CTCIF3	CGIF3	CTEIF2	CHTIF2	CTCIF2	CGIF2	CTEIF1	CHTIF1	CTCIF1	CGIF1					
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x008	DMA_CCR1	Reserved																	MEM2MEM	PL [1:0]		M SIZE [1:0]		PSIZE [1:0]		MINC	PINC	CIRC	DIR	TEIE	HTIE	TCIE	EN					
	Reset value																																	0	0	0	0	0
0x00C	DMA_CNDTR1	Reserved																	NDT[15:0]																			
	Reset value																												0	0	0	0	0	0	0	0	0	0
0x010	DMA_CPAR1	PA[31:0]																																				
	Reset value																																				0	0
0x014	DMA_CMAR1	MA[31:0]																																				
	Reset value																																				0	0
0x018	Reserved																																					

Registros del DMA

- DMA_CCR1
 - EN: Enable
 - TCIE: transf. comp. int. enable
 - HTIE: Half-transfer int. enable
 - TEIE: Transfer error int. enable
 - DIR: Periférico a memoria o al revés
 - CIRC: Modo circular
 - PINC: Incrementa puntero a periférico?
 - MINC: Incrementa puntero a memoria?
 - PSIZE[1:0] 1, 2 o 4 bytes
 - MSIZE[1:0] 1, 2 o 4 bytes
 - PL[1:0] Prioridad del canal (0 a 3)
 - MEM2MEM: Modo memoria a memoria
- DMA_CNDTR1
 - NDT[15:0]: Número de datos a transferir
- DMA_CPAR1 Registro de dirección de puerto de periférico
- DMA_CMAR1 Registro de dirección de puerto de memoria

Table 80. DMA register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x000	DMA_ISR	Reserved				TEIF7	HTIF7	TCIF7	GIF7	TEIF6	HTIF6	TCIF6	GIF6	TEIF5	HTIF5	TCIF5	GIF5	TEIF4	HTIF4	TCIF4	GIF4	TEIF3	HTIF3	TCIF3	GIF3	TEIF2	HTIF2	TCIF2	GIF2	TEIF1	HTIF1	TCIF1	GIF1	
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x004	DMA_IFCR	Reserved				CTEIF7	CHTIF7	CTCIF7	CGIF7	CTEIF6	CHTIF6	CTCIF6	CGIF6	CTEIF5	CHTIF5	CTCIF5	CGIF5	CTEIF4	CHTIF4	CTCIF4	CGIF4	CTEIF3	CHTIF3	CTCIF3	CGIF3	CTEIF2	CHTIF2	CTCIF2	CGIF2	CTEIF1	CHTIF1	CTCIF1	CGIF1	
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x008	DMA_CCR1	Reserved																	MEM2MEM	PL [1:0]	M SIZE [1:0]		P SIZE [1:0]		MINC	PINC	CIRC	DIR	TEIE	HTIE	TCIE	EN		
	Reset value																																0	0
0x00C	DMA_CNDTR1	Reserved																	NDT[15:0]															
	Reset value																												0	0	0	0	0	0
0x010	DMA_CPAR1	PA[31:0]																																
	Reset value																																	
0x014	DMA_CMAR1	MA[31:0]																																
	Reset value																																	
0x018	Reserved																																	

Representación en la HAL

```
typedef struct __DMA_HandleTypeDef
{
    DMA_Channel_TypeDef *Instance;
    DMA_InitTypeDef      Init;
    HAL_LockTypeDef       Lock;
    HAL_DMA_StateTypeDef  State;
    void                 *Parent;
    void (* XferCpltCallback)
        ( struct __DMA_HandleTypeDef * hdma);
    void(* XferHalfCpltCallback)
        ( struct __DMA_HandleTypeDef * hdma);
    void(* XferErrorCallback)
        ( struct __DMA_HandleTypeDef * hdma);
    void(* XferAbortCallback)
        ( struct __DMA_HandleTypeDef * hdma);
    __IO uint32_t          ErrorCode;
    DMA_TypeDef            *DmaBaseAddress;
    uint32_t               ChannelIndex;
} DMA_HandleTypeDef;
```

```
typedef struct
{
    __IO uint32_t ISR;
    __IO uint32_t IFCR;
} DMA_TypeDef;
```

```
typedef struct
{
    __IO uint32_t CCR;
    __IO uint32_t CNDTR;
    __IO uint32_t CPAR;
    __IO uint32_t CMAR;
} DMA_Channel_TypeDef;
```

```
typedef struct
{
    uint32_t Direction;
    uint32_t PeriphInc;
    uint32_t MemInc;
    uint32_t PeriphDataAlignment;
    uint32_t MemDataAlignment;
    uint32_t Mode;
    uint32_t Priority;
} DMA_InitTypeDef;
```

Ejemplo de DMA - Basado en ejemplo PWM sinusoide

Ejemplo sin DMA

- Se configura el PWM del Canal 1 del Timer 2 a 100 kHz
- Se guarda una sinusoide en un arreglo en memoria ram. Se guardan 100 muestras de manera de generarla a 1 kHz
- Se configura la interrupción del PWM
- El Procesador copia la muestra de la memoria al registro CCR1 del Tim2 en cada interrupción (actualizando el índice)

Con DMA

- Se configurará el PWM del Canal 1 del Timer 2 a 100 kHz
- Se guardará una sinusoide en un arreglo en memoria ram. Se guardaban 100 muestras de manera de generarla a 1 kHz
- Se configurará el DMA request del PWM
- El DMA copiará la muestra de la memoria al registro CCR1 del Tim2 en cada pedido. (Aprovecharemos características del DMA y no necesitaremos actualizar el índice)

Ejemplo de DMA - Configuración con MX

Timers

RTC

TIM1

☒ TIM2

TIM3

TIM4

Connectivity >

Computing >

Middleware >

☒ Parameter Settings

☒ User Constants

☒ NVIC Settings

☒ DMA Settings

☒ GPIO Settings

DMA Request	Channel	Direction	Priority
TIM2_CH1	DMA1 Channel 5	Memory To Peripheral	Low

Add

Delete

DMA Request Settings

Mode

Circular

Increment Address

☐

Data Width

Half Word

Peripheral

☐

Memory

☒

Byte

Ejemplo de DMA - Configuración en el código

hal_msp.c

```
void HAL_TIM_Base_MspInit (TIM_HandleTypeDef* htim_base)
{
    __HAL_RCC_TIM2_CLK_ENABLE ();

    hdma_tim2_ch1.Instance = DMA1_Channel5;
    hdma_tim2_ch1.Init.Direction =
DMA_MEMORY_TO_PERIPH;
    hdma_tim2_ch1.Init.PeriphInc = DMA_PINC_DISABLE;
    hdma_tim2_ch1.Init.MemInc = DMA_MINC_ENABLE;
    hdma_tim2_ch1.Init.PeriphDataAlignment =
DMA_PDATAALIGN_HALFWORD;
    hdma_tim2_ch1.Init.MemDataAlignment =
DMA_MDATAALIGN_BYTE;
    hdma_tim2_ch1.Init.Mode = DMA_CIRCULAR;
    hdma_tim2_ch1.Init.Priority = DMA_PRIORITY_LOW;
    HAL_DMA_Init (&hdma_tim2_ch1)
    __HAL_LINKDMA (htim_base, hdma[TIM_DMA_ID_CC1],
hdma_tim2_ch1);
}

// Luego en el main (MX_DMA_Init)
* DMA controller clock enable */
__HAL_RCC_DMA1_CLK_ENABLE();
```

hal_tim.c

```
HAL_StatusTypeDef HAL_TIM_PWM_Start_DMA(TIM_HandleTypeDef
*htim, uint32_t Channel, uint32_t *pData, uint16_t Length)
{
    /* Set the DMA compare callbacks */
    htim->hdma[TIM_DMA_ID_CC1]->XferCpltCallback =
TIM_DMADelayPulseCplt;
    htim->hdma[TIM_DMA_ID_CC1]->XferHalfCpltCallback =
TIM_DMADelayPulseHalfCplt;

    /* Set the DMA error callback */
    htim->hdma[TIM_DMA_ID_CC1]->XferErrorCallback =
TIM_DMAError ;

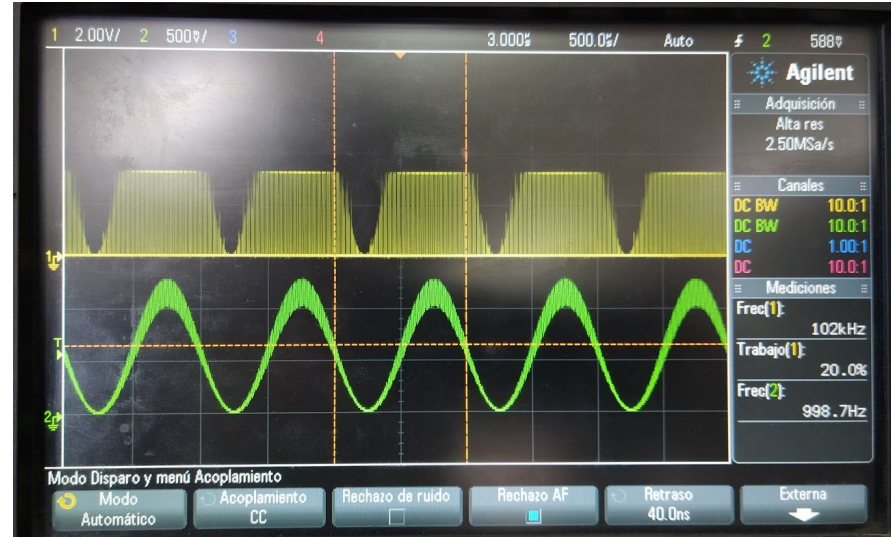
    /* Enable the DMA channel */
    HAL_DMA_Start_IT(htim->hdma[TIM_DMA_ID_CC1],
(uint32_t)pData, (uint32_t)&htim->Instance->CCR1,
Length)

    /* Enable the TIM Capture/Compare 1 DMA request */
    __HAL_TIM_ENABLE_DMA(htim, TIM_DMA_CC1);
}
```

Ejemplo de DMA - DMA y PWM

```
#include "main.h"
#include <math.h>
#define NSEN 100
#define VMAX 255
TIM_HandleTypeDef htim2;
DMA_HandleTypeDef hdma_tim2_ch1;
uint8_t onda[NSEN] = {0};
int idx=0;
int main(void){
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_TIM2_Init();

    /* USER CODE BEGIN 2 */
    for(idx = 0; idx < NSEN ; idx++){
        onda[idx] =
            (uint8_t)((float)VMAX/2.0*(sin(2.0*M_PI*idx/( float)NSEN)+1.0));
    }
    HAL_TIM_PWM_Start_DMA(&htim2, TIM_CHANNEL_1, (uint32_t*)onda,
NSEN);
    /* USER CODE END 2 */
    while (1) { }
}
```



Segundo ejemplo de DMA: disparo de request con otro periférico

- El timer 2 se configura para PWM

TIM2 Mode and Configuration

Mode

Slave Mode:

Trigger Source:

Clock Source:

Channel1:

Configuration

Parameter Settings

Configure the below parameters :

Counter Settings

Prescaler (PSC - 16 bits value): 0

Counter Mode: Up

Counter Period (AutoReload Register - 16 bits v...): 255

Internal Clock Division (CKD): No Division

auto-reload preload: Disable

Trigger Output (TRGO) Parameters

Master/Slave Mode (MSM bit): Disable (Trigger)

Trigger Event Selection: Reset (UG bit fr)

PWM Generation Channel 1

Mode: PWM mode 1

- El timer 3 para disparar un DMA request

DMA Settings

Configure the below parameters :

Counter Settings

Prescaler (PSC - 16 bits value): 0

Counter Mode: Up

Counter Period (AutoReload Register - 16 bits v...): 720

Internal Clock Division (CKD): No Division

auto-reload preload: Enable

DMA Request

DMA Request	Channel	Direction	Priority
TIM3_CH4/UP	DMA1 Channel 3	Memory To Peripheral	Low

DMA Request Settings

Mode:

Increment Address: ☐

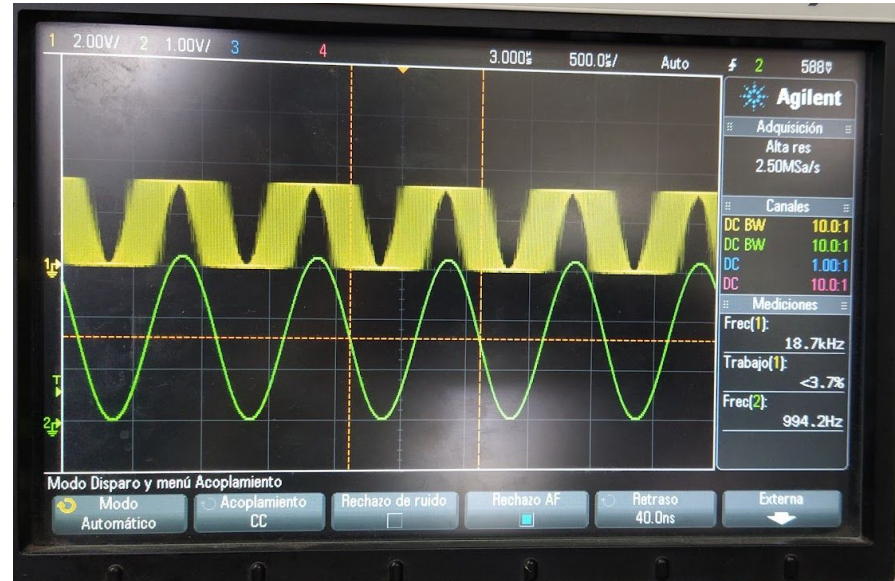
Data Width:

Peripheral: ☐ Memory: ☒

Segundo ejemplo de DMA: disparo de request con otro periférico

- Se da comienzo a los timers cada uno en su modo
- Se aprovecha el Request del timer 3 para realizar una transferencia de la memoria al periférico timer 2
- Como la función HAL_DMA_Start no activa el enable (DIER) debe hacerse “a mano”

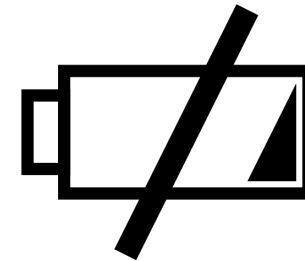
```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);  
  
HAL_DMA_Start(&hdma_tim3_ch4_up, (uint32_t) onda,  
(uint32_t) &(TIM2->CCR1), NSEN);  
__HAL_TIM_ENABLE_DMA(&htim3, TIM_DMA_UPDATE);  
  
HAL_TIM_Base_Start(&htim3);
```



Administración de energía y bajo consumo

¿Por qué bajo consumo?

- En algunas aplicaciones es obligatorio
 - Requerimiento de duración de baterías
 - Presupuesto energético o de potencia limitado
 - Disipación térmica
- En otras es deseable (“eficiencia energética”)
 - Mejora usabilidad
 - Evita sobredimensionar etapa de alimentación
 - Responsabilidad ambiental



Especificaciones de consumo

- En términos de energía

- Más familiar: baterías expresan su carga en mAh
 - Ejemplo: Batería de litio 800 mAh
 - Pasado a Coulomb ($[C] = [A][s]$)

$$800 \text{ mAh} \times 3600 \frac{s}{h} \times \frac{1}{1000} \frac{A}{mA} = 2880 \text{ C}$$

- Para conocer la corriente disponible debemos conocer la cantidad de tiempo
 - 20 Hs (0.05C) : 40 mA @ 3.7 V (C de C-rating)
- La energía depende de la tensión, que depende de la química de la pila
 - Li-ion 3.7V, alcalinas 1.5V, mercurio 1.35V
 - 3.7 V, 800 mAh

$$3.7 \text{ V} \times 800 \text{ mAh} = 2960 \text{ mWh} = 2.96 \text{ Wh} \equiv 2.96 \text{ Wh} \times 3600 \frac{s}{h} = 10656 \text{ J}$$

- En términos de potencia

- Corriente máxima
 - Ejemplo: Fuente con suministro máximo de 600 mA
 - Tasa de descarga de las baterías
- Podemos “engañar” a la fuente usando un filtro para picos de duración relativa corta (preocupación usual en inalámbricos)

Variación de consumo en circuitos digitales

- En sistemas digitales la potencia está en relación con la capacitancia de los nodos, la tensión (al cuadrado) y la frecuencia

$$P = C \times V^2 \times f$$

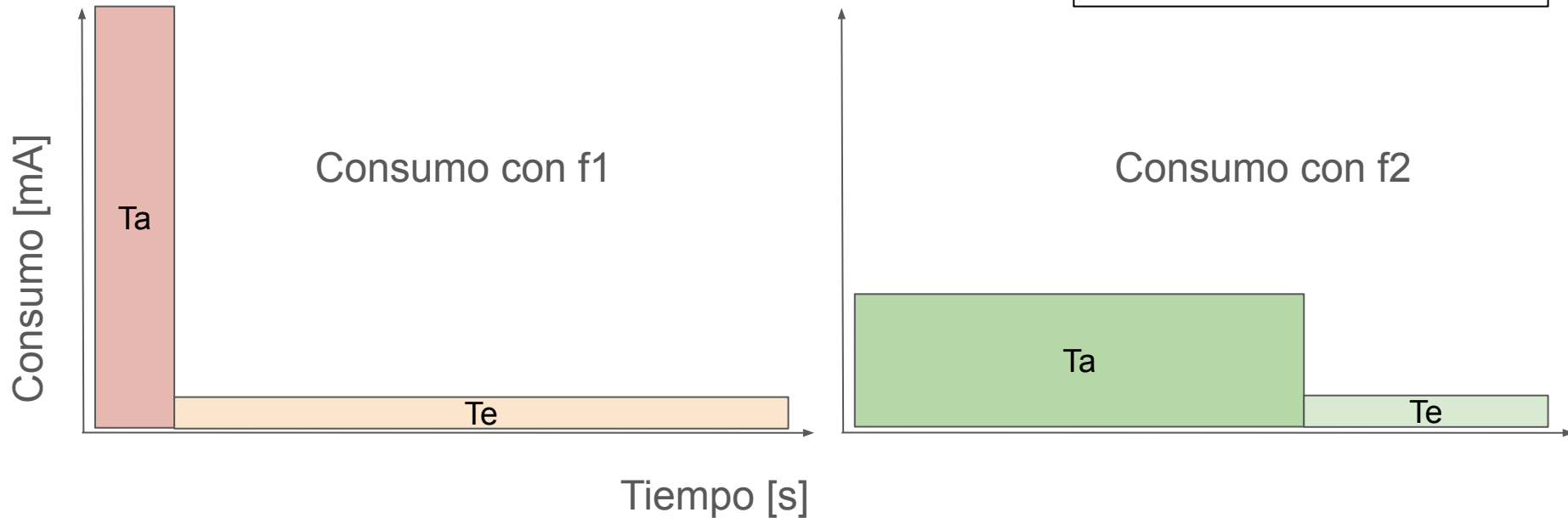
- El tiempo de ejecución T para N instrucciones resulta de los ciclos por instrucción y la frecuencia:

$$T = \frac{N \times CPI}{f}$$

- La energía consumida es por lo tanto

$$E = T \times P = C \times V^2 \times N \times CPI$$

¿Dependencia con la frecuencia?



Dependencia con la frecuencia



- Ejemplo: Un programa ejecuta 5000 ciclos de un lazo for y cambia la frecuencia de clock AHB (y derivados)

Table 9. General operating conditions

Symbol	Parameter	Conditions	Min	Max	Unit
f_{HCLK}	Internal AHB clock frequency	-	0	72	MHz
f_{PCLK1}	Internal APB1 clock frequency	-	0	36	
f_{PCLK2}	Internal APB2 clock frequency	-	0	72	

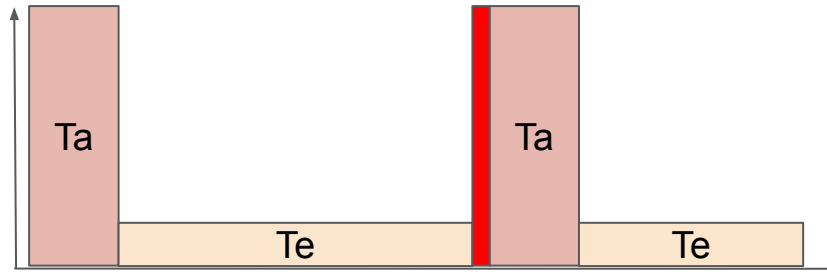
- En el microcontrolador, otros consumos aportan al total

Estrategias de administración del consumo

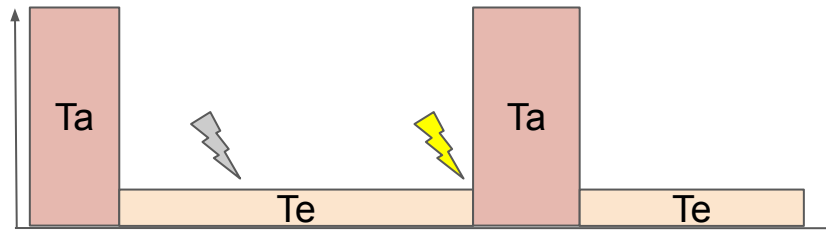
- La **tensión de alimentación** afecta el consumo, pero no siempre es un parámetro que podemos variar (e.g. el STM32F103 tiene un regulador interno)
- La **frecuencia de operación** afecta el consumo pero, teóricamente, la energía utilizada será la misma, y en la práctica, vemos que debemos investigar más para saber si ahorraremos o no
- Las estrategias de bajo consumo más usuales pasan por **apagar todo lo que no sea necesario**, incluyendo el procesador

Relaciones de compromiso

- Entre la “**profundidad del apagado**” y la **velocidad de respuesta a eventos** (porque debemos esperar a encender lo que se apagó)



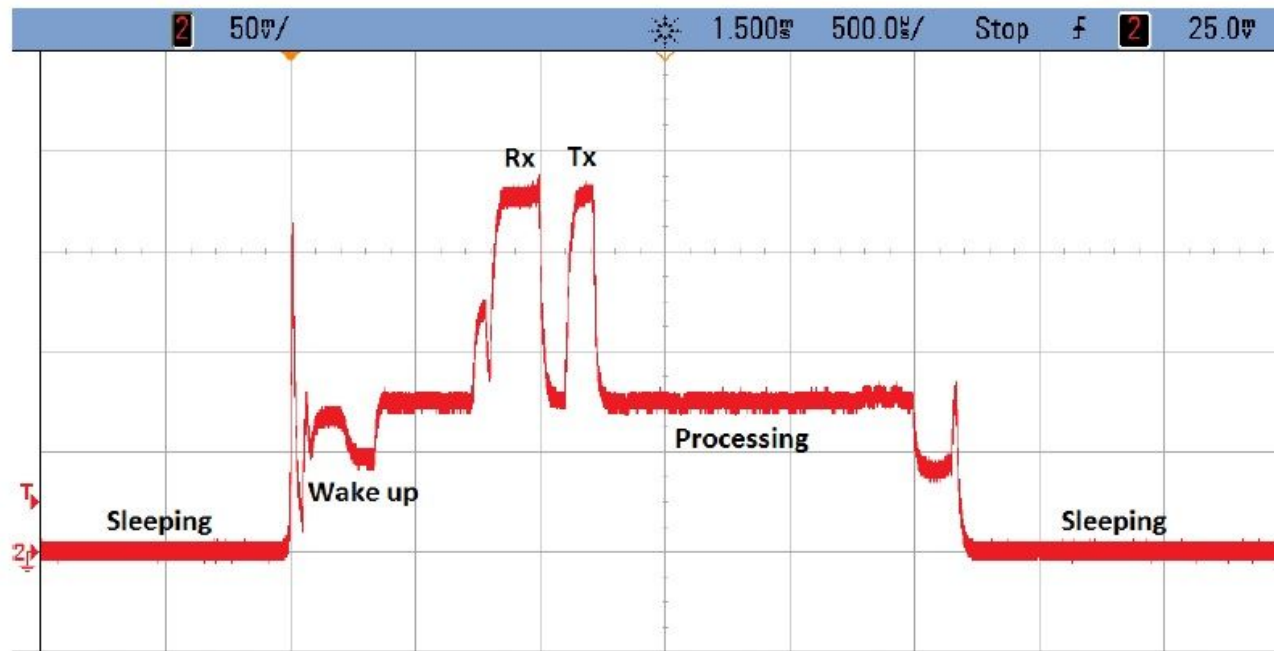
- Recursos disponibles** al apagar subsistemas



Modos de bajo consumo

- Para administrar las relaciones de compromiso se crean “perfiles” de bajo consumo
- Los distintos perfiles permiten distintos puntos de relación entre consumo y velocidad de respuesta
- Los microcontroladores más orientados a bajo consumo presentan mayor flexibilidad en estos modos (más modos, mayor granularidad en opciones)
- Los más comunes: Sleep, Standby
- El microcontroladores multi-núcleo, pueden incluir núcleos de bajo consumo para mantener la actividad y despertar al “hermano mayor” cuando sea necesario (e.g. ESP32)

Perfil de potencia



Perfil de potencia de un posible sensor BLE

Benchmarks

- Al ser tan variables los perfiles, puede ser difícil comparar entre microcontroladores al momento de dimensionar un sistema
- Recurrimos a los benchmarks como primera aproximación
 - DMIPS = iteraciones Dhrystone/1757 (Dhrystone de la VAX-11/780, primera máquina de 1 MIPS)
 - DMIPS/MHz Refleja la organización de la máquina
 - $(\text{DMIPS/MHz}) \times (\text{MHz/mWatt}) = \text{DMIPS/mWatt}$

<https://www.eembc.org/coremark/scores.php>

CoreMark[®]

An EEMBC Benchmark

	Compiler	Execution Memory	MHz	CoreMark	CoreMark / MHz [↑]	DMIPS/MHz
Atmel AT89C51RE2 in X2 mode (6 clocks/machine cycle)	Keil C51 v8.18	Internal RAM & flash (static)	22	2.36	0.11	
Microchip PIC18F46K22	Microchip MPLAB XC8 v1.32	Code in Flash, Data in RAM	64	7.23	0.11	
Atmel ATmega644	avr-gcc-4.3.2	Flash & SRAM 20 MHz (Static)	20	10.81	0.54	0.33
Texas Instruments MSP430F5438	Code Composer Studio 4.1.2	Internal flash and RAM (static)	18	11.10	0.62	0.31
Microchip PIC24FJ64GA004	gcc 4.0.3	Code Flash. Data SRAM (Static)	32	23.87	0.75	0.50
STMicroelectronics STM32F103	GCC 4.4.1	Internal Flash	72	108.26	3.32	1.25

Ejemplo de cálculo

- Cada 1 minuto el microcontrolador debe realizar un cálculo de aproximadamente un millón de instrucciones y luego pasar a stand-by.

a) ¿Cuánto durará una batería de 2400 mAh si se utiliza la siguiente CPU (llamémoslo CPU08)?

0.125 DMIPS/MHz

$I_{act} = 1 \text{ mA/MHz}$, 3V

$I_{stand-by} = 16 \text{ uA}$, 3V

$f_{max} = 8 \text{ MHz}$

Frec. 8MHz: $8 \text{ MHz} * 0.125 \text{ DMIPS/MHz} = 1 \text{ DMIPS}$ $T_{act} = 1 \text{ s}$ $I_{act} = 8 \text{ mA}$

Frec. 4MHz: $4 \text{ MHz} * 0.125 \text{ DMIPS/MHz} = 0.5 \text{ DMIPS}$ $T_{act} = 2 \text{ s}$ $I_{act} = 4 \text{ mA}$

$I_{media, 8MHz} = (1 \text{ s} * 8 \text{ mA} + 59 \text{ s} * 0.016 \text{ mA}) / 60 = 0.15 \text{ mA}$

Por lo tanto la duración de la batería estará dada aproximadamente por :

$2400 \text{ mAh} / 0.15 \text{ mA} = 16000 \text{ horas} = \mathbf{1.8 \text{ años}}$

Ejemplo de cálculo

- Cada 1 minuto el microcontrolador debe realizar un cálculo de aproximadamente un millón de operaciones y luego pasar a stand-by.

b) ¿Cuánto durará la misma batería si se utiliza un ATmega328P con AVR?

- 0.33 DMIPS/MHz (AVR)
- Active mode: 1.5mA 3V - 4MHz
- Power-down mode: 1µA 3V

$$@4MHz \quad 4 MHz * 0.33 DMIPS/MHz = 1.32 DMIPS \quad T_{act} = 0.75 s \quad I_{act} = 1.5 mA$$

$$I_{media} = (0.75 s * 1.5 mA + 59 s * 0.001 mA) / 60 = 0.02 mA$$

$$Duración de la batería: 2400 mAh / 0.02 mA = \mathbf{14 \text{ años}}$$

c) ¿Cuánto duraría la batería si se mantuviera siempre activo? **d)** ¿Y si se mantuviera siempre en stand-by?

$$c) I_{media} = 4 (08) \text{ ó } 1.5 mA (AVR)$$

$$Duración de la batería: \mathbf{24 \text{ días (08) } 66 \text{ días (AVR)}}$$

$$d) I_{media} = 0.016 (08) \text{ ó } 0.001 mA (AVR)$$

$$Duración de la batería: \mathbf{17 \text{ años (08) } 274 \text{ años (AVR) } > \text{RETENCIÓN BATERÍA}}$$

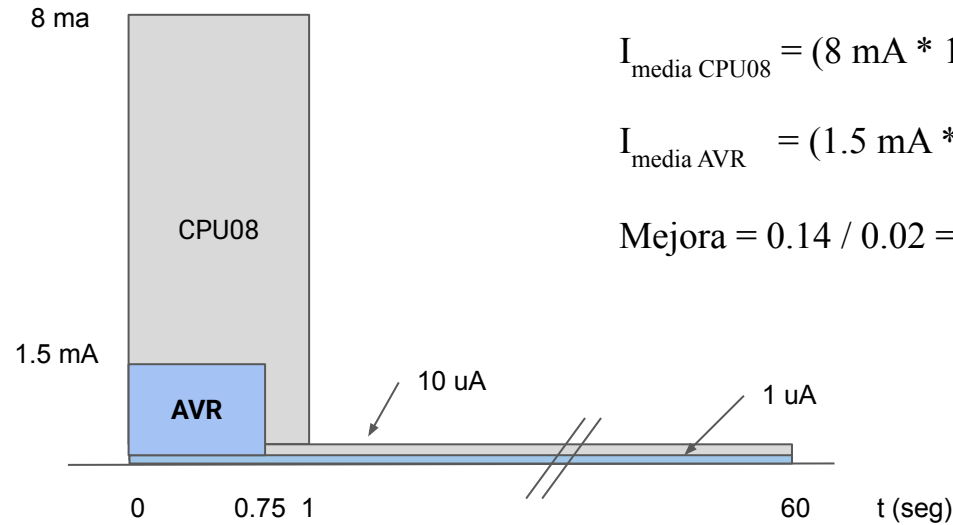
$$T_{\text{CPU08}} = 1 / (0.125 \text{ DMIPS/MHz} * 8 \text{ MHz}) = 1 \text{ s}$$

$$T_{\text{AVR}} = 1 / (0.33 \text{ DMIPS/MHz} * 4 \text{ MHz}) = 0.75 \text{ s}$$

$$I_{\text{media CPU08}} = (8 \text{ mA} * 1 \text{ s} + 10 \text{ uA} * 60 \text{ s}) / 60 \text{ s} = 0.14 \text{ mA}$$

$$I_{\text{media AVR}} = (1.5 \text{ mA} * 0.75 \text{ s} + 1 \text{ uA} * 60 \text{ s}) / 60 \text{ s} = 0.02 \text{ mA}$$

$$\text{Mejora} = 0.14 / 0.02 = \mathbf{7x}$$



Control de consumo en el STM32F103

- Primera herramienta: manejo de clocks

5.3.1 Slowing down system clocks

In Run mode the speed of the system clocks (SYSCLK, HCLK, PCLK1, PCLK2) can be reduced by programming the prescaler registers. These prescalers can also be used to slow down peripherals before entering Sleep mode.

For more details refer to [Section 7.3.2: Clock configuration register \(RCC_CFGR\)](#).

5.3.2 Peripheral clock gating

In Run mode, the HCLK and PCLKx for individual peripherals and memories can be stopped at any time to reduce power consumption.

To further reduce power consumption in Sleep mode the peripheral clocks can be disabled prior to executing the WFI or WFE instructions.

Modos de bajo consumo del procesador Cortex-M3

- Cada microcontrolador basado en procesadores ARM Cortex-M tendrá al menos tres Power Modes: Run, Sleep y Deep Sleep.
 - El modo **Run** es cuando el procesador está completamente encendido.
 - El modo **Sleep** detendrá el reloj de la CPU pero dejará operativos el reloj del sistema, la memoria flash y los periféricos. Volver al modo Run toma en el orden de ciclos de instrucción
 - El modo **Deep Sleep** no sólo detendrá el reloj de la CPU, sino que también apagará el reloj del sistema, la memoria flash y el PLL. Volver al modo Run toma en el orden de varios microsegundos.
- El fabricante del microcontroladores tiene la capacidad de personalizar sus módulos de bajo consumo (ST divide el Deep Sleep en Stop y Standby).
- Se accede a los modos a partir de una configuración en el registro SCB más una instrucción WFI o WFE

Configuración del modo de bajo consumo

- > preface
- > Introduction
- > The Cortex-M3 Processor
- > The Cortex-M3 Instruction Set
- > Cortex-M3 Peripherals
 - About the Cortex-M3 peripherals
 - > Nested Vectored Interrupt Controller
 - > System control block
 - Auxiliary Control Register
 - CPUID Base Register
 - Interrupt Control and State Register
 - Vector Table Offset Register
 - Application Interrupt and Reset Control...
 - System Control Register**
 - Configuration and Control Register
 - System Handler Priority Registers
 - System Handler Control and State Register
 - Configurable Fault Status Register
 - HardFault Status Register
 - MemManage Fault Address Register
 - BusFault Address Register
 - Auxiliary Fault Status Register
 - System control block usage hints and tips
- > System timer, SysTick

System Control Register

The SCR controls features of entry to and exit from low power state. See the register summary in Table 4.12 for its attributes. The bit assignments are:



[4] SEVONPEND	Send Event on Pending bit: 0 = only enabled interrupts or events can wakeup the processor, disabled interrupts are excluded 1 = enabled events and all interrupts, including disabled interrupts, can wakeup the processor. When an event or interrupt enters pending state, the event signal wakes up the processor from WFE. If the processor is not waiting for an event, the event is registered and affects the next WFE. The processor also wakes up on execution of an SEV instruction or an external event.
[3] -	Reserved.
[2] SLEEPDEEP	Controls whether the processor uses sleep or deep sleep as its low power mode: 0 = sleep 1 = deep sleep.
[1] SLEEPONEXIT	Indicates sleep-on-exit when returning from Handler mode to Thread mode: 0 = do not sleep when returning to Thread mode 1 = enter sleep, or deep sleep, on return from an ISR. Setting this bit to 1 enables an interrupt driven application to avoid returning to an empty main application.

Instrucciones para entrar en modos de bajo consumo

- **Wait For Interrupt (WFI):**

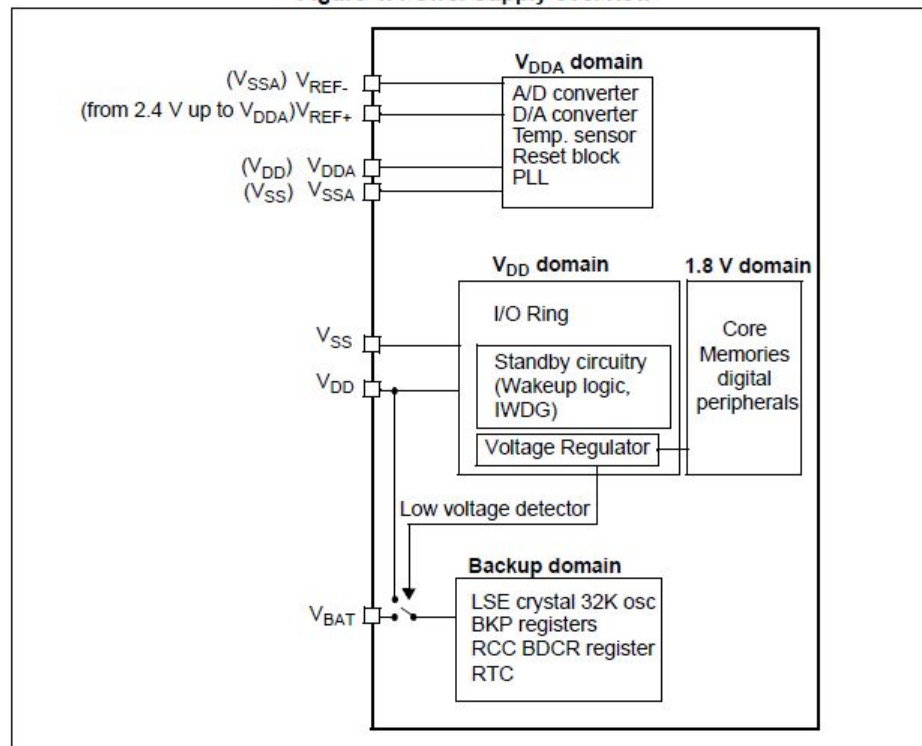
- “Sleep incondicional” que detiene la ejecución inmediatamente.
- Se vuelve a despertar con interrupciones (dependiendo de que estén habilitadas, y por supuesto de “cuán dormido” esté).
- Si había una interrupción pendiente, entra al modo sleep, pero lo despierta.

- **Wait For Event (WFE):**

- El despertar depende no sólo de interrupciones sino también de eventos.
- Los eventos son similares a interrupciones pero pueden provenir de distintas fuentes (son como “otras interrupciones” que se rutean en un bus aparte en el procesador).
- WFE se despierta por interrupciones aunque estén enmascaradas (porque generan eventos) además de sumar eventos que no son interrupciones
- WFE no entra al modo de bajo consumo si había un evento pendiente
- Según la configuración, puede evitar el mecanismo de atención de interrupción y ser más rápido que WFI

Dominios de alimentación

Figure 4. Power supply overview



Uniendo todo: Modos de bajo consumo en el STM32F103

Table 11. Low-power mode summary

Mode name	Entry	Wakeup	Effect on 1.8V domain clocks	Effect on V _{DD} domain clocks	Voltage regulator
Sleep (Sleep now or Sleep-on -exit)	WFI	Any interrupt	CPU clock OFF	None	ON
	WFE	Wakeup event	no effect on other clocks or analog clock sources		
Stop	PDDS and LPDS bits + SLEEPDEEP bit + WFI or WFE	Any EXTI line (configured in the EXTI registers)	All 1.8V domain clocks OFF	HSI and HSE oscillators OFF	ON or in low-power mode (depends on <i>Power control register (PWR_CR)</i>)
Standby	PDDS bit + SLEEPDEEP bit + WFI or WFE	WKUP pin rising edge, RTC alarm, external reset in NRST pin, IWDG reset			OFF

Modo 1 : Sleep mode

- Es el modo que más consume, ya que sólo se dejan de suministrar pulsos de clock al procesador, pero se mantiene energizado.
- Toda actividad que no dependa del procesador, puede continuar
- Como se preserva el estado de la memoria (RAM, registros), puede continuar la ejecución inmediatamente al suministrar clock al procesador

Table 12. Sleep-now

Sleep-now mode	Description
Mode entry	WFI (Wait for Interrupt) or WFE (Wait for Event) while: – SLEEPDEEP = 0 and – SLEEPONEXIT = 0 Refer to the Cortex®-M3 System Control register.
Mode exit	If WFI was used for entry: Interrupt: Refer to Section 10.1.2: Interrupt and exception vectors If WFE was used for entry Wakeup event: Refer to Section 10.2.3: Wakeup event management
Wakeup latency	None

Table 13. Sleep-on-exit

Sleep-on-exit	Description
Mode entry	WFI (wait for interrupt) while: – SLEEPDEEP = 0 and – SLEEPONEXIT = 1 Refer to the Cortex®-M3 System Control register.
Mode exit	Interrupt: refer to Section 10.1.2: Interrupt and exception vectors .
Wakeup latency	None

Modo 2 : Stop mode

- Comparado con el modo Sleep, el procesador se apaga junto con todo lo que depende del dominio de 1.8 V
- Pero el regulador en si no se apaga, sino que se pone en modo low-power, para suministrar energía a la RAM y registros
- También se preserva el estado de los pines de entrada/salida
- Se reducen las fuentes de eventos o interrupciones para despertar al microcontrolador, ya que muchos periféricos dejan de funcionar

Table 14. Stop mode

Stop mode	Description
Mode entry	WFI (Wait for Interrupt) or WFE (Wait for Event) while: – Set SLEEPDEEP bit in Cortex®-M3 System Control register – Clear PDDS bit in Power Control register (PWR_CR) – Select the voltage regulator mode by configuring LPDS bit in PWR_CR Note: To enter Stop mode, all EXTI Line pending bits (in Pending register (EXTI_PR)), all peripheral interrupt pending bits, and RTC Alarm flag must be reset. Otherwise, the Stop mode entry procedure is ignored and program execution continues.
Mode exit	If WFI was used for entry: Any EXTI Line configured in Interrupt mode (the corresponding EXTI Interrupt vector must be enabled in the NVIC). Refer to Section 10.1.2: Interrupt and exception vectors . If WFE was used for entry: Any EXTI Line configured in event mode. Refer to Section 10.2.3: Wakeup event management
Wakeup latency	HSI RC wakeup time + regulator wakeup time from Low-power mode

Modo 3: Standby mode

- Es el modo de consumo mínimo: podemos pensar que se apaga todo por completo, salvo lo que se alimenta del dominio de Backup
- Esto deja fuentes de eventos acotadas para despertarlo:
 - pines WKUP o NRST
 - Alarma del RTC
 - Watchdog independiente IWDG
- Los pines quedan en estado de alta impedancia
- Volver del deepsleep es lo mismo que un reset, y lo único que se preserva es
 - Un bit SBF para saber si venimos del deepsleep
 - Datos que hayamos guardado en la memoria de backup

Table 15. Standby mode

Standby mode	Description
Mode entry	WFI (Wait for Interrupt) or WFE (Wait for Event) while: – Set SLEEPDEEP in Cortex®-M3 System Control register – Set PDDS bit in Power Control register (PWR_CR) – Clear WUF bit in Power Control/Status register (PWR_CSR) – No interrupt (for WFI) or event (for WFI) is pending
Mode exit	WKUP pin rising edge, RTC alarm event's rising edge, external Reset in NRST pin, IWDG Reset.
Wakeup latency	Reset phase

I/O states in Standby mode

In Standby mode, all I/O pins are high impedance except:

- Reset pad (still available)
- TAMPER pin if configured for tamper or calibration out
- WKUP pin, if enabled

Debug mode

By default, the debug connection is lost if the application puts the MCU in Stop or Standby mode while the debug features are used. This is due to the fact that the Cortex®-M3 core is no longer clocked.

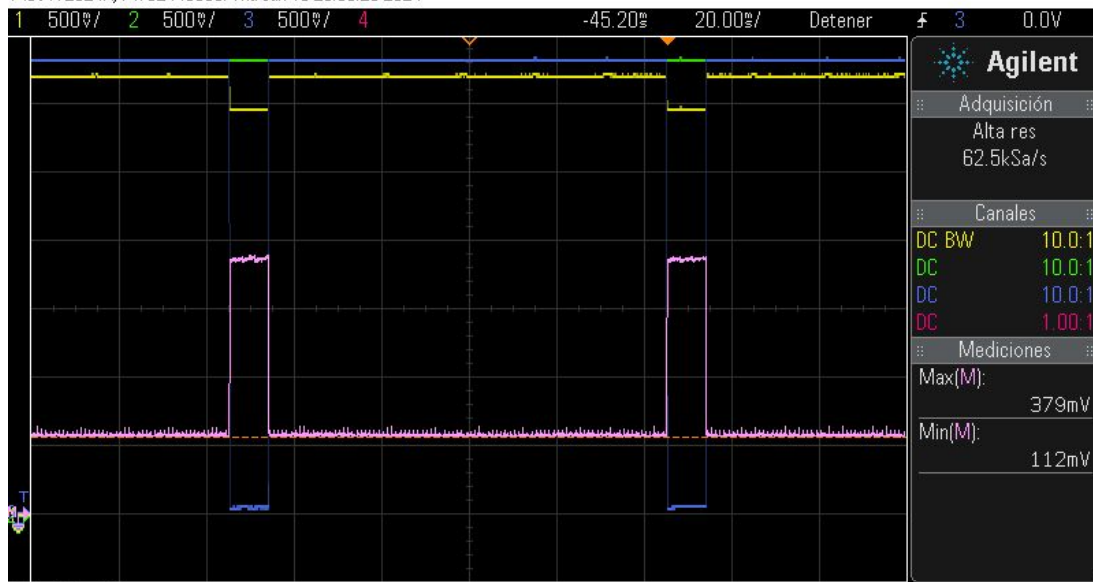
Ejemplo utilizando la HAL: **Sleep on Exit**

```
/* main */
HAL_TIM_Base_Start_IT(&htim2);
HAL_PWR_EnableSleepOnExit();
while (1)
{
}

/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_RESET);
    for(int i=0; i<50000; i++) ;
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_SET);
}
/* USER CODE END 4 */
```


Ejemplo utilizando la HAL: **Sleep on Exit**

MSO-X 2024A, MY52440880: Thu Jun 13 20:59:20 2024



Consumo en modo Run: 37.9 mA

Consumo en modo Sleep: 11.2 mA

Ejemplo utilizando la HAL: **Sleep on Exit**

MSO-X 2024A, MY52440880: Thu Jun 13 21:07:40 2024



Pero qué son esos “pelitos”?

Ejemplo utilizando la HAL: **Sleep on Exit**

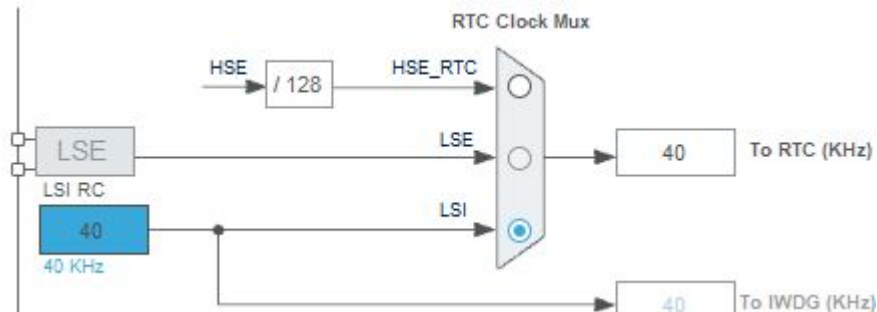


```
/* main */  
HAL_TIM_Base_Start_IT(&htim2);  
HAL_SuspendTick();  
HAL_PWR_EnableSleepOnExit();  
while (1)  
{  
  
/* USER CODE BEGIN 4 */  
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){  
    HAL_ResumeTick();  
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_RESET);  
    for(int i=0; i<50000; i++) ;  
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_SET);  
    HAL_SuspendTick();  
}  
/* USER CODE END 4 */
```

Ejemplos utilizando la HAL: **Stadby + RTC**

RTC Mode and Configuration

Mode

☒ Activate Clock Source☐ Activate CalendarRTC OUT RTC Output on the Tamper pin☒ Tamper

```

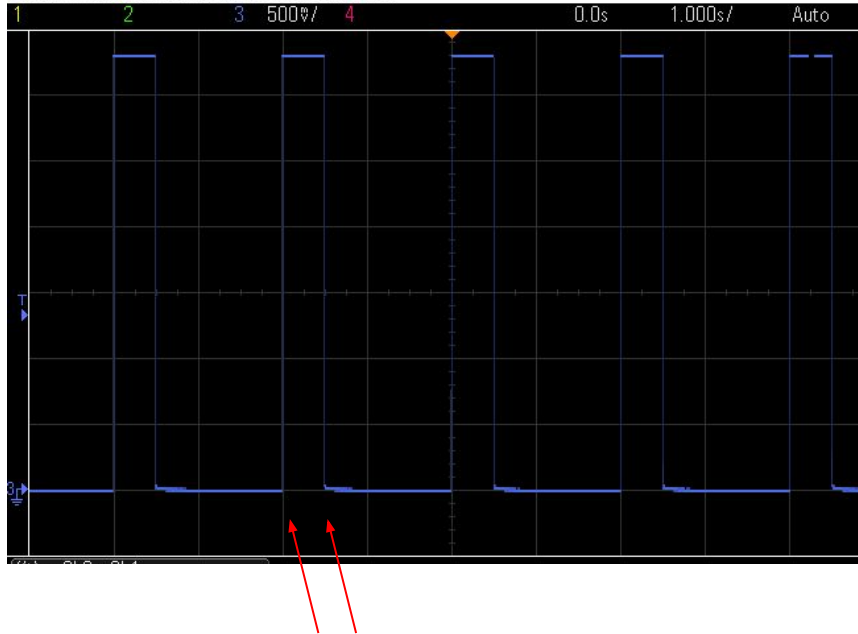
RTC_TimeTypeDef myTime;
RTC_AlarmTypeDef myAlarm;

main{
    /* USER CODE BEGIN 2 */
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_SET);
    HAL_Delay(500);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_RESET);
    HAL_RTC_GetTime(&hrtc, &myTime, RTC_FORMAT_BIN);
    myTime.Seconds += 1;
    myAlarm.AlarmTime = myTime;
    HAL_RTC_SetAlarm(&hrtc, &myAlarm, FORMAT_BIN);
    __HAL_PWR_CLEAR_FLAG(PWR_FLAG_WU);
    HAL_PWR_EnterSTANDBYMode();
    /* USER CODE END 2 */
    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {
    }
    }

```

Ejemplo utilizando la HAL: **Stadby + RTC**

MSO-X 2024A, MY52440880: Fri Jun 14 06:55:11 2024



```

RTC_TimeTypeDef myTime;
RTC_AlarmTypeDef myAlarm;
main{
/* USER CODE BEGIN 2 */
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5,GPIO_PIN_SET);
HAL_Delay(500);
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5,GPIO_PIN_RESET);
HAL_RTC_GetTime(&hrtc, &myTime, RTC_FORMAT_BIN);
myTime.Seconds += 1;
myAlarm.AlarmTime = myTime;
HAL_RTC_SetAlarm(&hrtc, &myAlarm, FORMAT_BIN);
__HAL_PWR_CLEAR_FLAG(PWR_FLAG_WU);
HAL_PWR_EnterSTANDBYMode();
/* USER CODE END 2 */
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
}
}

```